

Posterior Security: Anonymity and Message Hiding of Standard Signatures

Tsz Hon Yuen
john.tszhonyuen@monash.edu
Monash University
Clayton, Victoria, Australia

Ying-Teng Chen
ying-teng.chen@monash.edu
Monash University
Clayton, Victoria, Australia

Shimin Pan
smpan@connect.hku.hk
The University of Hong Kong
Pokfulam, Hong Kong

Jiangshan Yu
jiangshan.yu@sydney.edu.au
The University of Sydney
Sydney, NSW, Australia

Joseph K. Liu
joseph.liu@monash.edu
Monash University
Clayton, Victoria, Australia

ABSTRACT

We introduce *posterior security* of digital signatures, the additional security features *after* the original signature is generated. It is motivated by the scenario that some people store their secret keys in secure hardware and can only obtain a standard signature through a standardized interface. In this paper, we consider two different posterior security features: anonymity and message hiding.

We first introduce *incognito signature*, a new mechanism to anonymize a standard signature. Different from other ring or group signatures, the signer generates a standard (non-anonymous) signature first. The signature is then anonymized by a converter before sending to the verifier, by hiding the signer public key with a set of decoy public keys. We then introduce *concealed signature* which hides the message in a commitment. The standard signature is converted such that it can be verified with the commitment. The models of *posterior anonymity* and *posterior message hiding* capture the separation of the signer and the converter. Anonymity or message hiding is provided by the converter *after* the creation of a standard signature by the signer.

We give generic constructions of incognito signature and concealed signature. It can be applied to standard signatures like Schnorr. It gives the first practical anonymized ECDSA signature, and the signature size is logarithmic to the number of decoy public keys n . The existing ring signature scheme with ECDSA keys is at least 152 times longer than our scheme for $n \leq 4096$.

The incognito signature and concealed signature can be composed to provide posterior anonymity and message hiding. It is useful in applications like two-tier central bank digital currency, where users want to hide their addresses (public keys) and transaction amounts (messages) when the payment is settled in the interbank layer.

KEYWORDS

Privacy, signatures

1 INTRODUCTION

Digital signatures are a cornerstone of secure communication and authentication. As their adoption grows in real-world systems like cryptocurrency, a critical question arises: Can we enhance the privacy and flexibility of digital signatures *after* they are already

generated? This leads us to the concept of *posterior security*: security properties that are added after a standard signature is produced.

Consider two practical scenarios:

- **Hardware Security Module (HSM):** Organizations often use HSMs to securely store cryptographic keys. These modules typically support only standard signing algorithms (e.g., those compliant with FIPS 186-2 standard), offering no room to add advanced security or privacy features at signing time.
- **Digital wallet custodian:** In the context of cryptocurrencies, users may delegate key management to third-party custodians. These services take great pains to avoid the keys being exfiltrated. While this increases security, users lose the flexibility to add security or privacy features to their transactions.

In such scenarios, existing primitives like group signatures or ring signatures fall short. Group signatures and ring signatures provide anonymity to the signer. Group signatures require the signer to join a group before signature generation, and ring signatures provide user privacy during signature generation. Both primitives require the modification of the signing algorithm, and hence are not applicable to the two scenarios above. Neither allows for non-interactive post hoc privacy transformations. Posterior security addresses this gap: it allows the signer to produce a standard signature using unmodified signing logic, and a separate party — the *converter*, who does not have access to the signing key — to later enhance the security or privacy of the signature.

Posterior security could be useful in different settings:

- The secret key is not accessible by the owner (e.g., when using an HSM or a digital wallet custodian), so the owner can only apply additional protection to the signature after it is generated.
- The additional security or privacy feature is not required at the time of signature generation, but becomes necessary afterward.
- A fast message signing (using the standard signature) is needed (e.g., on a mobile phone), and the additional feature can be added by the converter (e.g., a server) afterward.

1.1 Motivations

To illustrate the benefits of posterior security, consider a two-tier Central Bank Digital Currency (CBDC) [3] architecture proposed

jointly by the Bank for International Settlements and the Hong Kong Monetary Authority. In this architecture, users sign transactions using standard digital wallets on their mobile devices in the first tier. These signatures are later submitted to commercial banks for settlement on an interbank ledger in the second tier.

A key design challenge in this setting is reconciling user privacy with regulatory compliance. On one hand, users must generate standard, non-anonymous signatures so that commercial banks can retain the ability to audit and report suspicious transactions for anti-money laundering and other regulatory purposes. On the other hand, the interbank settlement layer should not expose users' spending behaviors, such as who spent what amount, to other banks or observers of the shared ledger.

At the same time, commercial banks are not trusted with users' private signing keys, both for security reasons and to respect the user's control over their own transactions. Instead, users hold their own keys and produce signatures independently. As a result, any transformation to protect user privacy (such as anonymizing public keys or hiding message content) must occur after the signature is generated, without modifying the signature logic or accessing the secret key.

This motivates the need for post hoc privacy enhancements, i.e., mechanisms that can be applied after the standard signature is produced. Posterior security addresses this requirement by allowing the commercial bank to apply protections before publishing transactions to the second-tier ledger, with the following features:

- **Posterior Anonymity:** The user's public key can be anonymized by the commercial bank, protecting the spender's identity from third parties during second-tier interbank settlement.
- **Posterior Message Hiding:** The transaction amount (message) can be hidden in a cryptographic commitment, ensuring that only authorized parties can access this sensitive information during settlement or compliance checks.

In this scenario, the user is the signer and the commercial bank acts as the converter. Posterior security also provides some other unique advantages: (1) **Efficiency:** the user's digital wallet only generates a standard signature, which is extremely fast during payment. Most heavy computation is done by the commercial bank's server afterward. This separation of signer and converter improves the user experience. (2) **Compatibility:** the digital wallet only outputs a standard signature, which implies minimal changes to the existing wallets, as well as the point-of-sales machines validating the signatures. It is also easier to integrate with existing hardware wallets. (3) **Compliance:** the commercial banks store all signed transactions. If the regulator wants to trace an illegal transaction on the ledger, the commercial bank can submit the original user signature as evidence. These three advantages cannot be achieved by existing privacy-preserving cryptocurrencies like Monero and ZCash.

1.2 Posterior Anonymity and Message Hiding

Motivated by the above CBDC application, this paper focuses on two types of posterior security: *posterior anonymity* and *posterior message hiding*.

Posterior anonymity ensures that the identity of the original signer remains anonymous to the verifier. It separates the duty of

signing a message and anonymizing a signature. It leads to some interesting applications.

- **Privacy-preserving signature with backward compatibility over existing key management systems.** In many commercial applications, secret keys are managed by secure hardware such as HSM. User can only access the key via standardized API, such as those signing algorithms defined in PKCS #11. For managing cryptocurrency wallet, many users rely on hardware wallet or third party wallet custodian service to manage their keys and to sign transactions. Posterior anonymity can work with those existing key management systems, by adding anonymity for the user without requiring any change to the core signing process.
- **Whistleblowing for non-signer.** In a parliament, an internal document is independently signed by some senators. One of the senator's secretary wants to be the whistleblower and to leak the document for public interest. However, the secretary cannot generate a ring signature by themselves because the secret key is owned by the senator. The secretary can use incognito signature to convert the original signature to an anonymous one. In this case, we need strong anonymity here because the secretary does not want the senator (the real signer) to find out who leak the document.

Posterior message hiding ensures that the original message is concealed from the verifier. It also has some useful applications.

- **Sanitizing data for sharing.** HL7 FHIR is a global standard for health data and is used in electronic health records systems. The health record is signed by a doctor for authenticity at the end of a medical consultation. Later on, if a hospital is obliged to report excerpts of their patients medical data to an authority for disease surveillance, the excerpts should be signed by the doctors. Yet some parts of personal data in the excerpts should be sanitized by the hospital staff. Concealed signature can be used with a commitment scheme that supports partial opening.

The benefits of combining posterior anonymity and posterior message hiding are not limited to CBDC.

- **User-control anonymous credentials.** Traditional anonymous credentials can only show that a user obtains a credential from an organization, and the user is hidden within all members of the organization. Incognito signature allows the user to extend the level of anonymity in anonymous credentials. Consider credit bureaus using the industry standard *Metro 2* to generate credit reports. Alice obtains a signed report with a message "Alice credit score = 710" from a financial institution. Alice can convert it to an incognito concealed signature with a set of public keys from multiple financial institutions, to show that she has a high credit score without revealing the specific institution and the exact score. Alice chooses the set of public keys from globally recognised financial institutions, to demonstrate that her creditworthiness is acknowledged by reputable sources. Alice commits her score as $C = \text{Commit}(710)$. Alice can further show that C is a commitment of a number greater than 700 by using a range proof, without revealing her actual credit score. It

ensures her privacy and protects her from potential biases or targeted financial marketing.

In this paper, we define two primitives, incognito signature and concealed signature, to model posteriors anonymity and posterior message hiding, respectively. These two properties can be achieved simultaneously in an incognito concealed signature.

1.3 Modelling Posterior Anonymity for Incognito Signature

Incognito signature is defined upon a standard signature. Consider a standard signature scheme with Setup (defining system parameters), KeyGen (generating a secret key sk and a public key pk), Sign (signing a message m using sk to obtain a signature σ) and Verify (verifying a signature σ with respect to m and pk). By our definition of posterior security, Setup, KeyGen, Sign and Verify remain the same.

In order to achieve posterior anonymity, we define another verification algorithm CVerify for incognito signature to be run upon a set of n public keys $\mathbf{pk} = \{pk_1, \dots, pk_n\}$, like the verification of ring signatures. However, it is different from ring signatures that we introduce a new Convert algorithm that turns σ into an incognito signature σ_I . The modified verification algorithm will validate σ_I with respect to m and the public key set $\mathbf{pk}$.

Considering our motivation from the HSM and two-tier CBDC, the Convert algorithm should not have access to the secret key and should be run by a *converter* different from the owner of the key. Therefore, we define the Convert algorithm to take $\sigma, m, \mathbf{pk}$ and the original signer public key pk as input (with $pk \in \mathbf{pk}$), and to output the incognito signature σ_I . To the best of the authors' knowledge, the concept of separating the role of the key owner and the role of the *converter* is new to the literature. These two roles are the same party for ring signatures and group signatures. In contrast, blind signature separates the role of the key owner and the message owner. Threshold signatures delegate the role of the key owner to multiple parties.

Next, we define the security model of posterior anonymity. Similar to the anonymity model of ring signatures, the adversary should not be able to identify the real signer pk from $\mathbf{pk}, m$ and σ_I with probability larger than $1/n$. However, the anonymity model is different from ring signatures because of the separation of roles. In particular, there are two types of anonymity:

- (1) Anonymous with respect to non-signers. It is useful for the HSM example. The signature is generated from a hardware device, and the user does not care about anonymity with respect to the hardware device. It could lead to a more efficient instantiation.
- (2) Strong anonymity against the signer. It means that even the signer cannot determine if σ_I is generated from the signature he created before. This strong anonymity is similar to the blindness property of blind signature, in which the signed message is hidden with respect to the signer. It is useful for the whistleblowing example described later.

Generic Construction. In this paper, we give a generic construction of incognito signatures. It can be efficiently instantiated to

ECDSA and other standardized signature schemes. We give the high level idea below.

We can write the verification function as an equality:

$$\text{Verify}(\sigma, m, pk) = 1. \quad (1)$$

The above equality holds if $(sk, pk) \leftarrow \text{KeyGen}()$ and $\sigma \leftarrow \text{Sign}(m, sk)$. In order to anonymize σ , it means that we have to hide the pk in Eq. 1. Suppose that $\mathbf{pk} = \{pk_1, \dots, pk_n\}$ is a set of n public keys, where pk_j is the original public key pk and j is an arbitrary index in $[1, n]$. Define a binary vector $\vec{b} = (b_1, \dots, b_n)$, where $b_i = 0$ for all $i \neq j$ and $b_j = 1$. Then, we can convert the equation 1 to $\text{Verify}(\sigma, m, \prod_{i=1}^n pk_i^{b_i}) = 1$. In order to maintain the anonymity, we have to hide pk , the vector $\vec{b}$ as well as σ . If we have a non-interactive zero-knowledge argument of knowledge (NIZKAoK) π for the following relation:

$$\{(m, \mathbf{pk}; \sigma, pk, \vec{b}) : \text{Verify}(\sigma, m, pk) = 1 \quad \wedge \quad pk = \prod_{i=1}^n pk_i^{b_i} \quad \wedge \quad \vec{b} \in \{0, 1\}^n \quad \wedge \quad \|\vec{b}\|_1 = 1\},$$

where $\|\vec{b}\|_1$ is the ℓ_1 -norm of $\vec{b}$. Then we can use π as the incognito signature. The verifier only has to validate the NIZKAoK π with respect to the message m and the set of public keys $\mathbf{pk}$. The unforgeability of the generic incognito signature is based on the soundness of the NIZKAoK and the unforgeability of the underlying signature scheme. The posterior anonymity of the generic incognito signature is based on the zero-knowledge property of the NIZKAoK.

1.4 Modelling Posterior Message Hiding for Concealed Signature

To hide the message while preserving the validity of a standard signature, we propose to conceal the message by using a commitment scheme with algorithms (Com, Decom). In order to achieve posterior message hiding, we introduce a new Convert algorithm that turns σ into a concealed signature σ_{CS} and outputs a trapdoor aux . The modified verification algorithm will validate σ_{CS} with respect to the public key pk . There is a new Decom algorithm to decommit σ_{CS} to a message m using aux .

Next, we define the security model of concealed signatures. Posterior message hiding means that a concealed signature should contain no information of the original message m . Apart from the classical unforgeability model of signatures, our unforgeability model also captures the case that the adversary should not be able to convert the standard signature to a concealed signature which is valid and can be decommitted to a different message $m' \neq m$.

Generic Construction. In this paper, we give a generic construction of concealed signatures. It can be efficiently instantiated to the Boneh-Boyen signatures. We give the high level idea below.

In order to hide σ , it means that we have to hide the message m in Eq. 1 using a commitment C . Consider that the algorithm $\text{Com}(m)$ outputs a commitment C and an auxiliary information aux , such that $\text{Decommit}(m, C, aux) = 1$. If we have a NIZKAoK π for the following relation:

$$\{(pk, C; \sigma, m, aux) : \text{Verify}(\sigma, m, pk) = 1 \wedge \text{Decom}(m, C, aux) = 1\},$$

then we can use (π, C) as the concealed signature. The verifier only has to validate π with respect to the commitment C and the public key pk . The unforgeability of the generic concealed signature is based on the soundness of the NIZKAoK, the binding property of the commitment scheme and the unforgeability of the underlying signature scheme. The posterior message hiding of the generic concealed signature is based on the zero-knowledge property of the NIZKAoK and the hiding property of the commitment scheme.

1.5 Related works

Signatures with Modified Signing Algorithm. In this paper, we investigate posterior security which keeps the original signing algorithm of a standard signature. In the literature, there are a number of researches working on the opposite: keeping the original verification algorithm and modifying the signing algorithm. A blind signature [12] scheme converts the signing algorithm into an interactive protocol between a signer (who owns the secret key) and a user (who knows the message to be signed). The standard signature is output by the user and the signer is not able to link the signature with the past signing sessions. Threshold signatures [15] allow splitting the secret key to a group of users in the *key generation phase*, so that a subgroup of them is able to output a standard signature through an interactive signing protocol. These cryptographic primitives are more common in the Schnorr-signature setting. ECDSA-based constructions, compatible with the existing ECDSA-based blockchain verifiers, are more complex and challenging to construct (e.g., [20, 30]).

Signatures with Anonymity Property. Chuam [13] proposed the use of *anonymous credentials* to allow users to be authenticated, without leaking their identity. It starts from an issuer generating a credential for the user. Then the user communicate with the verifier, so that the verifier can only learn that he is talking to a legitimate user. Many variants of anonymous credentials were proposed, e.g., credentials are associated with multiple attributes and users selectively choose which attributes to reveal, or show that the attributes satisfy some policies.

Group signature [14] allows a member to prove his membership of a group anonymously, with the additional opening features that enables an opening authority to reveal the identity of the real signer. It introduces the accountability feature to signatures with anonymity.

Ring signature [31] does not require the issuance of credential. Ring signature is associated with a set of public keys *chosen by the signer*, and the signer proves that he owns the secret key of one of the public key in the set. Ring signature has full anonymity, such that no one can reveal the identity of the signer. Variants of ring signatures include linkable ring signature, in which ring signature generated from the same signer are linked, without compromising the identity of the signer.

Recent researches study the wide spectrum between full anonymity to full accountability [6, 27, 29, 33]. To the best of the authors' knowledge, there is no previous work considering the separation of the signing process and the anonymizing process as the incognito signature does. Although we only consider full anonymity in the current description of incognito signature, different levels of

anonymity or accountability ([6, 27, 29]) may also be applied to incognito signature as well. We leave this as a future work.

Anonymizing ECDSA. Most generic ring signature schemes are not compatible with ECDSA. The first ring signature [31] is based on trapdoor one-way function like RSA or Rabin. The AOS ring signature [1] can be built from a Hash-then-One-Way type (such as RSA) or Three-move type (such as Schnorr) signatures. DualRing [34] is a generic ring signature with efficient instantiation from a special type of three-move identification. ECDSA does not work with these schemes.

To the best of the authors' knowledge, the only work that gives a concrete construction of group or ring signatures for ECDSA keys is zkAttest [17]. They presented a Σ -protocol to verify an ECDSA signature with a committed public key. When working with Groth-Kohlweiss membership proof [21], they derived a ring signature and a group signature scheme. The proof involves the mapping from group elements to the (x, y) coordinates. As a result, the corresponding zero-knowledge proof is quite complex and uses a specially chosen Tom curve. The ECDSA-based ring signature size is at least 197KB (see Table 1), which is not practical.

Conceptually, the construction of zkAttest is similar to ECDSA instantiation of our generic construction. However, the authors of [17] did not consider the separation between the signer and the converter. In contrast, they consider the signature conversion by a non-signer as a "multiprotocol attack" which cannot be captured by the anonymity model of ring signatures.

Message Modification. Sanitizable signature [2] allows authorized semi-trusted party to modify parts of a signed message in a limited and controlled manner, without interacting with the signer.

A chameleon-hash function [24] is a hash function with a trapdoor which allows one to find a collision of the message input. The trapdoor owner can be different from the one who computes the hash [11]. When used in a signature scheme with hashing, the message can be modified by the trapdoor owner after the signature generation.

1.6 Our Contributions

- (1) We propose the concept of *posterior security*, and demonstrate that posterior anonymity and posterior message hiding can be achieved after a standard signature is generated.
- (2) We define a new cryptographic primitive called *incognito signature* with the security models of posterior anonymity and unforgeability.
- (3) We define a new cryptographic primitive called *concealed signature*. We define the security models of posterior message hiding and unforgeability.
- (4) We give generic constructions of incognito signature and concealed signature. They can be applied to different discrete-logarithm (DL) based, RSA-based, pairing-based and lattice-based signatures.
- (5) We give the *first* efficient ECDSA-based signature with anonymity property. Compared with zkAttest [17], our instantiation is 337 $\times$ shorter (resp. 216 $\times$, 152 $\times$) for a ring size of $n = 2$ (resp. $n = 64$, $n = 4096$).

2 PRELIMINARIES

In this paper, we use λ as the security parameter and an arrow such as $\vec{a}$ to represent a vector or matrix. We use $\circ$ to define Hadamard products: $\vec{a} \circ \vec{b} := (a_1 b_1, \dots, a_n b_n)$. For inner products, it is represented as $\langle \vec{a}, \vec{b} \rangle = \sum_{i=1}^n a_i b_i$. For vectors, we also define $\vec{a}^{\vec{b}} := (a_1^{b_1}, \dots, a_n^{b_n})$. We also define consecutive powers: $\vec{y}^n := (1, y, y^2, \dots, y^{n-1})$.

Zero-Knowledge Argument of Knowledge (ZKAoK). A prover can convince a verifier that some statement holds without revealing any information.

An argument consists of three PPT algorithms $(\mathcal{K}, \mathcal{P}, \mathcal{V})$, which are CRS (Common Reference String) generator $\mathcal{K}$, the prover $\mathcal{P}$ and the verifier $\mathcal{V}$. A CRS τ is produced by $\mathcal{K}$ on input 1^λ and a transcript tr is produced by $\mathcal{P}$ and $\mathcal{V}$ on inputs s and t , which is denoted by $tr \leftarrow \langle \mathcal{P}(s), \mathcal{V}(t) \rangle$. We write $\langle \mathcal{P}(s), \mathcal{V}(t) \rangle = b$ to denote that the verifier $\mathcal{V}$ accepts $b = 1$ or rejects $b = 0$. We define the language:

$$\mathcal{L}_\tau = \{u \mid \exists w : (\tau, u, w) \in \mathcal{R}\},$$

where w is a witness and u is a set of statements u in the relation $\mathcal{R}$. An argument of knowledge $(\mathcal{K}, \mathcal{P}, \mathcal{V})$ should satisfy perfect completeness and statistical witness-extended emulation. The properties of completeness, statistical witness-extended emulation and special honest-verifier zero-knowledge are given in the appendix due to the space limit.

Signature. We review the notion of standard signature.

- **Setup.** On input a security parameter 1^λ , it outputs a public parameters param . The public parameters are omitted from all algorithms' input in this paper for simplicity.
- **KeyGen.** It outputs a secret key sk and a public key pk .
- **Sign.** On input a message m and a secret key sk , it outputs a signature σ .
- **Verify.** On input a signature σ , a message m and a public key pk , it output 1 for valid or 0 for invalid.

The common security model for signature is the existential unforgeability against chosen message attack (EU-CMA). In the security game, a challenger $\mathcal{C}$ runs $\text{param} \leftarrow \text{Setup}(1^\lambda)$ and $(\text{sk}, \text{pk}) \leftarrow \text{KeyGen}()$. An adversary $\mathcal{A}$ is given param, pk , and a signing oracle access to obtain signatures on any message m . The signature scheme is EU-CMA secure if no probabilistic polynomial time (PPT) adversary $\mathcal{A}$ can output a pair of valid message m^* and signature σ^* (where m^* was not queried to the signing oracle).

3 POSTERIOR ANONYMITY: INCOGNITO SIGNATURE

As discussed in Section 1, the incognito signature is verified against a set of public keys, by converting from the verification of a standard signature ($\text{Verify}(\sigma, m, \text{pk}) = 1$) to the verification of an incognito signature ($\text{CVerify}(\sigma_I, m, \text{pk}) = 1$). It is possible that the public keys in pk are defined by a set of public parameters param . If this is the case, then the verification of the incognito signature becomes $\text{Verify}(\text{param}, \sigma_I, m, \text{pk}) = 1$. As a result, the construction also needs to hide the real signer's param in the set of param . For simplicity, we only consider the case of a single public parameters in this paper.

3.1 Security Models

We define the notion of an incognito signature as a standard signature (Setup, KeyGen, Sign, Verify) with the following algorithms.

- **IS.Setup.** On input the public parameters param of the standard signature, it outputs the incognito public parameters param_I .
- **IS.Convert.** On input a standard signature σ for a message m , a public key pk^* and a set of public key pk (with $\text{pk}^* \in \text{pk}$), it (1) outputs $\perp$ if $\text{Verify}(\sigma, m, \text{pk}^*) = 0$ or (2) outputs an incognito signature σ_I .
- **IS.CVerify.** On input an incognito signature σ_I , a message m and a set of public key pk , it output 1 for valid or 0 for invalid.

Correctness of the incognito signature means that for all messages m in the message space, $(\text{sk}^*, \text{pk}^*) \leftarrow \text{KeyGen}()$, $\sigma \leftarrow \text{Sign}(m, \text{sk}^*)$, all public keys in pk are in the key space such that $\text{pk}^* \in \text{pk}$, we have:

$$\text{IS.CVerify}(\text{IS.Convert}(\sigma, m, \text{pk}^*, \text{pk}), m, \text{pk}) = 1.$$

Strong posterior anonymity against full key and signature exposure. We define a *strong posterior anonymity*, so that the incognito signature remains anonymous even if all secret keys and the standard signatures are known to the adversary. It is formally defined in the following game between a challenger $\mathcal{C}$ and an adversary $\mathcal{A}$.

- (1) The challenger $\mathcal{C}$ runs $\text{param} \leftarrow \text{Setup}(1^\lambda)$ and $\text{param}_I \leftarrow \text{IS.Setup}(\text{param})$. $\mathcal{C}$ runs $(\text{sk}_i, \text{pk}_i) \leftarrow \text{KeyGen}()$ using randomness ω_i for $i \in [1, n]$ and sets $\text{pk} = (\text{pk}_1, \dots, \text{pk}_n)$. $\mathcal{C}$ sends $\text{param}, \text{param}_I, \text{pk}$, and $\{\omega_i\}_{i=1}^n$ to the adversary $\mathcal{A}$.
- (2) $\mathcal{A}$ computes sk_{i_0, i_1} from ω_{i_0, i_1} and returns a message m^* , two distinct indices $i_0, i_1 \in [1, n]$, a set of public keys pk^* for which $\text{pk}_{i_0}, \text{pk}_{i_1} \in \text{pk} \cap \text{pk}^*$, and $\sigma_0^* \leftarrow \text{Sign}(m^*, \text{sk}_{i_0})$, $\sigma_1^* \leftarrow \text{Sign}(m^*, \text{sk}_{i_1})$.
- (3) $\mathcal{C}$ picks a random bit b and runs $\sigma_I^* \leftarrow \text{IS.Convert}(\sigma_b^*, m^*, \text{pk}_{i_b}, \text{pk}^*)$. $\mathcal{C}$ returns σ_I^* to $\mathcal{A}$.
- (4) $\mathcal{A}$ outputs a bit b^* .

If $b^* = b$, then $\mathcal{A}$ wins the game. We say that an incognito signature is *strongly posterior anonymous against full key and signature exposure* if no PPT adversary can win the game with non-negligible probability over $1/2$.

In the model, we allow $\mathcal{A}$ to calculate σ_b^* for $b = 0/1$ in step 2. In step 3, $\mathcal{C}$ converts one of the σ_b^* into σ_I^* . Hence the strong posterior anonymity guarantees that the incognito signature σ_I^* remains anonymous even when the original signature is given and all keys are known to the adversary.

Posterior anonymity against full key exposure. We also propose a weaker notion of posterior anonymity, so that the adversary does not know the standard signature used to create the incognito signature σ_I^* . It is formally defined in the following game between a challenger $\mathcal{C}$ and an adversary $\mathcal{A}$.

- (1) The challenger $\mathcal{C}$ runs $\text{param} \leftarrow \text{Setup}(1^\lambda)$ and $\text{param}_I \leftarrow \text{IS.Setup}(\text{param})$. $\mathcal{C}$ runs $(\text{sk}_i, \text{pk}_i) \leftarrow \text{KeyGen}()$ using randomness ω_i for $i \in [1, n]$ and sets $\text{pk} = (\text{pk}_1, \dots, \text{pk}_n)$. $\mathcal{C}$ sends $\text{param}, \text{param}_I$ and pk to the adversary $\mathcal{A}$.

- (2) $\mathcal{A}$ returns a message m^* , two distinct indices $i_0, i_1 \in [1, n]$ and a set of public keys $\mathbf{pk}^*$ for which $\mathbf{pk}_{i_0}, \mathbf{pk}_{i_1} \in \mathbf{pk} \cap \mathbf{pk}^*$.
- (3) C picks a random bit b and runs $\sigma^* \leftarrow \text{Sign}(m^*, \mathbf{sk}_{i_b})$. C runs $\sigma_I^* \leftarrow \text{IS.Convert}(\sigma^*, m^*, \mathbf{pk}_{i_b}, \mathbf{pk}^*)$. C returns $\{\omega_i\}_{i=1}^n$ and σ_I^* to $\mathcal{A}$.
- (4) $\mathcal{A}$ outputs a bit b^* .

If $b^* = b$, then $\mathcal{A}$ wins the game. We say that an incognito signature is *posterior anonymous against full key exposure* if no probabilistic polynomial time (PPT) adversary can win the game with non-negligible probability over $1/2$. It provides a similar level of anonymity provided by ring signatures [4], such that the incognito signature is anonymous even if the adversary can obtain all signing keys.

Unforgeability with respect to insider corruption. Apart from the standard unforgeability definition of signatures, we also include a user corruption oracle as in ring signature [4], and a new *conversion oracle* for incognito signature. It models the case that given an incognito signature on a message m and a set of public keys $\mathbf{pk}'$, the adversary cannot create (1) another incognito signature on m for a different set of public key $\mathbf{pk}^*$ and (2) another incognito signature on the public keys $\mathbf{pk}'$ for a different message m^* .

Unforgeability of incognito signature is formally defined in the following game between a challenger C and an adversary $\mathcal{A}$.

- (1) The challenger C runs $\text{param} \leftarrow \text{Setup}(1^\lambda)$ and $\text{param}_I \leftarrow \text{IS.Setup}(\text{param})$. C runs $(\mathbf{sk}_i, \mathbf{pk}_i) \leftarrow \text{KeyGen}()$ for $i \in [1, n]$ and sets $\mathbf{pk} = (\mathbf{pk}_1, \dots, \mathbf{pk}_n)$. C sends param , param_I , $\mathbf{pk}$ to the adversary $\mathcal{A}$.
- (2) $\mathcal{A}$ can make adaptive queries to oracles:
 - Signing oracle: on input a message m , a public key $\mathbf{pk}_i \in \mathbf{pk}$, it returns a signature $\sigma \leftarrow \text{Sign}(m, \mathbf{sk}_i)$.
 - Corruption oracle: on input a public key $\mathbf{pk}_i$, it returns the corresponding secret key $\mathbf{sk}_i$.
 - Conversion oracle: on input a message m , a set of public key $\mathbf{pk}'$, and a public key $\mathbf{pk}_i \in \mathbf{pk} \cap \mathbf{pk}'$, it returns an incognito signature $\sigma_I \leftarrow \text{IS.Convert}(\text{Sign}(m, \mathbf{sk}_i), m, \mathbf{pk}_i, \mathbf{pk}')$.
- (3) $\mathcal{A}$ returns a subset of public keys $\mathbf{pk}^* \subseteq \mathbf{pk}$, an incognito signature σ_I^* and a message m^* to C .

$\mathcal{A}$ wins the game if $\text{IS.CVerify}(\sigma_I^*, m^*, \mathbf{pk}^*) = 1$ and there is no signing oracle query with input $(m^*, \mathbf{pk}_i)$ such that $\mathbf{pk}_i \in \mathbf{pk}^*$, no corruption oracle query with input $\mathbf{pk}_i \in \mathbf{pk}^*$, and no conversion oracle query with input $(m^*, \mathbf{pk}^*, \mathbf{pk}_i)$.

We say that an incognito signature is *unforgeable with respect to insider corruption* if no PPT adversary can win the game with non-negligible probability.

3.2 Generic Construction of Incognito Signature

Given a standard signature scheme (Setup , KeyGen , Sign , Verify) and a non-interactive zero-knowledge argument of knowledge $(\mathcal{K}, \mathcal{P}, \mathcal{V})$, we give a generic construction of incognito signature.

IS.Setup. The system parameters param_I includes the parameters $\tau \leftarrow \mathcal{K}(1^\lambda)$.

IS.Convert. On input a signature σ , a message m , a public key $\mathbf{pk}^*$, and a set of public keys $\mathbf{pk} = \{\mathbf{pk}_1, \dots, \mathbf{pk}_n\}$, with $\mathbf{pk}_j = \mathbf{pk}^*$ for

some index $j \in [1, n]$, it generates a NIZKAoK $\pi \leftarrow \mathcal{P}(\tau, u, w)$ for the following relation:

$$\{u := (m, \mathbf{pk}); w := (\sigma, \mathbf{pk}, \vec{b}) : \text{Verify}(\sigma, m, \mathbf{pk}) = 1 \wedge \mathbf{pk} = \prod_{i=1}^n \mathbf{pk}_i^{b_i} \wedge (\vec{b} - \vec{1}^n) \circ \vec{b} = \vec{0}^n \wedge \vec{b} \cdot \vec{1}^n = 1\}.$$

It outputs the incognito signature $\sigma_I = \pi$.

IS.CVerify. On input an incognito signature $\sigma_I = \pi$, a message m and a set of public key $\mathbf{pk}$, it outputs 1 if $1 \leftarrow \mathcal{V}(\tau, u, \pi)$ and outputs 0 otherwise.

THEOREM 3.1. *The generic incognito signature is unforgeable with respect to insider corruption if the underlying signature scheme is unforgeable, and the NIZKAoK has witness extended emulation.*

PROOF. **Setup.** The simulator $\mathcal{S}$ receives the system parameters param and a public key $\mathbf{pk}^*$ from the challenger C of the standard signature. $\mathcal{S}$ generates param_I using the parameters $\tau \leftarrow \mathcal{K}(1^\lambda)$. $\mathcal{S}$ generates $n - 1$ key pairs using KeyGen and obtains a set of n public keys $\mathbf{pk}$ with $\mathbf{pk}^* \in \mathbf{pk}$. $\mathcal{S}$ sends param , param_I and $\mathbf{pk}$ to the adversary $\mathcal{A}$ attacking the generic incognito signature.

Oracle Simulation. When $\mathcal{A}$ asks the signing oracle for a message m and a public key $\mathbf{pk}_i \neq \mathbf{pk}^*$, $\mathcal{S}$ uses the corresponding secret key $\mathbf{sk}_i$ to sign and returns the signature σ . If $\mathcal{A}$ asks for a signature on a message m and a public key $\mathbf{pk}^*$, $\mathcal{S}$ forwards the query to its challenger C to answer.

When $\mathcal{A}$ asks the corruption oracle for a public key $\mathbf{pk}_i \neq \mathbf{pk}^*$, $\mathcal{S}$ returns the corresponding secret key $\mathbf{sk}_i$. If $\mathcal{A}$ asks the corruption oracle for $\mathbf{pk}^*$, $\mathcal{S}$ declares failure and exits.

When $\mathcal{A}$ asks the conversion oracle for a message m , a set of public key $\mathbf{pk}'$, and a public key $\mathbf{pk}_i \in \mathbf{pk} \cap \mathbf{pk}'$, $\mathcal{S}$ returns $\sigma_I \leftarrow \text{IS.Convert}(\text{Sign}(m, \mathbf{sk}_i), m, \mathbf{pk}_i, \mathbf{pk}')$ if $\mathbf{pk}_i \neq \mathbf{pk}^*$. If $\mathbf{pk}_i = \mathbf{pk}^*$, then $\mathcal{S}$ runs the oracle $\mathcal{O}$ of the NIZKAoK to generate $\sigma_I = \pi$.

Forgery. Finally $\mathcal{A}$ returns a subset of public keys $\mathbf{pk}^* \subseteq \mathbf{pk}$, an incognito signature σ_I^* and a message m^* , so that $\text{IS.Verify}(\sigma^*, m^*, \mathbf{pk}^*) = 1$. Denote $n^* = |\mathbf{pk}^*|$. By the witness extended emulation property of the NIZKAoK, $\mathcal{S}$ can extract $\sigma^*, \mathbf{pk}_j^*, \vec{b}, \vec{a}$ so that:

$$\vec{a} \circ \vec{b} = (\vec{b} - \vec{1}^{n^*}) \circ \vec{b} = \vec{0}^{n^*}.$$

It implies that $\vec{b}$ is a binary vector. Since $(\vec{b} - \vec{1}^{n^*}) \cdot \vec{b} = 1$, it implies that only one element in $\vec{b}$ is equal to one and the others are equal to zero. It implies that for some $j \in [1, n^*]$,

$$\text{Verify}(\sigma^*, m^*, \mathbf{pk}_j^*) = 1, \quad \mathbf{pk}_j^* \in \mathbf{pk}^*.$$

If $\mathbf{pk}_j^* \neq \mathbf{pk}^*$, $\mathcal{S}$ declares failure and exits. Since $\mathbf{pk}_j^* \in \mathbf{pk}^* \subseteq \mathbf{pk}$ and $|\mathbf{pk}| = n$, the probability of $\mathbf{pk}_j^* = \mathbf{pk}^*$ is at least $1/n$. Since there was no signing oracle query with input $(m^*, \mathbf{pk}^*)$ and no conversion oracle query with input $(m^*, \mathbf{pk}^*, \mathbf{pk}_j)$, $\mathcal{S}$ returns (σ^*, m^*) as the forgery of the underlying signature scheme.

If a PPT adversary can break the unforgeability of the incognito signature with non-negligible probability, $\mathcal{S}$ can also break the unforgeability of the underlying signature in polynomial time with non-negligible probability. Hence, we reach a contradiction. $\square$

THEOREM 3.2. *The generic incognito signature is strongly posterior anonymous against full key and signature exposure if the NIZKAoK has zero-knowledge property.*

PROOF. By the zero-knowledge property of the NIZKAoK, π does not leak the information of σ , pk as well as the signer index j . $\square$

4 INSTANTIATIONS OF INCOGNITO SIGNATURE

In this section, we will give efficient instantiations of the generic incognito signature. We consider the following types of digital signatures:

- For DL-based signatures without hashing the public key (e.g., Schnorr, ECDSA), we can use the Bulletproof [10] as the efficient instantiation of the NIZKAoK.
- For pairing-based signatures (e.g., BLS, BBS), we can use pairing-based ZK proof such as Dory [26] as the efficient instantiation of the NIZKAoK.
- For signatures in unknown order group (e.g., RSA-based or class-group-based), we can use the ZKAoK for unknown order group in [5].
- The signer's public key is included in the hash function input, such as the key-prefixed variants of Schnorr signature (e.g., EdDSA) and the lattice-based signature Dilithium. It requires a zk-SNARK for the hash function pre-image to hide the signer's public key.

4.1 DL-based: Incognito Schnorr Signature

We review the standard Schnorr signature [32] included in standards like ISO/IEC 14888-3.

Setup. On input a security parameter 1^λ , it outputs a cyclic group $\mathbb{G}$ of prime order p , a generator $g \in \mathbb{G}$ and a hash function $H : \{0, 1\}^* \times \mathbb{G} \rightarrow \mathbb{Z}_p$.

KeyGen. It picks a random secret key $\text{sk} = x$ and computes a public key $\text{pk} = g^x$.

Sign. To sign a message m with the secret key x , it picks a random $r \in \mathbb{Z}_p$ and computes $R = g^r$, $c = H(m, R)$, $z = r + cx$. It outputs the signature $\sigma = (c, z)$.

Verify. To verify a signature σ with a message m and a public key pk , it computes $R = g^z \text{pk}^{-c}$. It outputs 1 if $c = H(m, R)$ or 0 otherwise.

Incognito Schnorr Signature. Observe that the verification of the Schnorr signature involves a computation of $c = H(m, R = g^z \text{pk}^{-c})$. Following the generic construction, we have to use a zk-SNARK for the pre-image of a hash function. In order to avoid the complexity of such zk-SNARK, we give a different concrete instantiation with posterior anonymity with respect to non-signer (instead of strong posterior anonymity). In particular, the value R is included in the incognito Schnorr signature. The concrete construction is as follows.

IS.Setup. The system parameters param_I includes random independent generators $h, \vec{g} = (g_1, \dots, g_n), \vec{h} = (h_1, \dots, h_n)$ from $\mathbb{G}$, and hash functions $H_g : \mathbb{Z}_p \times \mathbb{G}^n \rightarrow \mathbb{G}$, $H_z : \mathbb{G}^2 \rightarrow \mathbb{Z}_p$, $H_w : \mathbb{G}^5 \rightarrow \mathbb{Z}_p^2$, $H_x : \mathbb{G}^2 \times \mathbb{Z}_p^2 \rightarrow \mathbb{Z}_p$ and $H_d : \mathbb{Z}_p^5 \rightarrow \mathbb{Z}_p$.

IS.Convert. On input a signature $\sigma = (c, z)$, a message m , a public key pk^* , and a set of public keys $\text{pk} = \{\text{pk}_1, \dots, \text{pk}_n\}$, with $\text{pk}_j = \text{pk}^*$ for some index $j \in [1, n]$, it computes $R = g^z \text{pk}_j^{-c}$ so that:

$$g^z = R \cdot \left(\prod_{i=1}^n \text{pk}_i^{b_i} \right)^c, \quad c = H(m, R),$$

where $b_j = 1$ and $b_i = 0$ for all $i \neq j$. In order to maintain anonymity, it needs a zero-knowledge proof π'_b for all b_i , as well as z and the index j . There are a few steps to fulfil this goal.

- (1) Commit pk_j . Compute a group element $g_0 = H_g(c, \text{pk})$. Pick a random $\beta \in \mathbb{Z}_p$ and compute a commitment $C_{\text{pk}} = g_0^\beta \text{pk}_j = g_0^\beta \text{pk}^{\vec{b}}$, where $\vec{b} = (b_1, \dots, b_n)$.
- (2) ZK Proof for z . Pick random $r_z, r_\beta \in \mathbb{Z}_p$ and compute $R_z = g^{r_z} g_0^{r_\beta c}$, $c_z = H_z(R_z, C_{\text{pk}})$, $s_z = r_z + c_z z$, $s_\beta = r_\beta + c_z \beta$. Denote $\pi_z = (R_z, s_z, s_\beta)$.
- (3) ZK Proof for $\vec{b}$. It generates a NIZKAoK π'_b for the vector $\vec{b}$. By using the Bulletproof [10], the proof size is logarithmic to n . Details are given in the appendix.

Finally, it outputs the incognito signature $\sigma_I = (R, C_{\text{pk}}, \pi_z, \pi'_b)$.

IS.CVerify. On input an incognito signature $\sigma_I = (R, C_{\text{pk}}, \pi_z, \pi'_b)$, a message m and a set of public key pk , it outputs 1 if all of the following checks hold and outputs 0 otherwise.

For $\pi_z = (R_z, s_z, s_\beta)$, it computes $c = H(m, R)$, $g_0 = H_g(c, \text{pk})$, $c_z = H_z(R_z, C_{\text{pk}})$, checks if

$$g^{s_z} g_0^{s_\beta c} = R_z R^{c_z} C_{\text{pk}}^{c \cdot c_z}.$$

The verification of the NIZKAoK π'_b is given in the appendix.

THEOREM 4.1. *The incognito Schnorr signature is posterior anonymous against full key exposure in the random oracle model.*

PROOF. It can be shown by the following lemmas.

LEMMA 4.2. π'_b is perfect honest-verifier zero-knowledge for the witness $\vec{b}$.

LEMMA 4.3. π_z is perfect honest-verifier zero-knowledge for the witness z .

PROOF. By picking random $c_z, s_z, s_\beta \in \mathbb{Z}_p$, we can compute $R_z = R^{c_z} C_{\text{pk}}^{c \cdot c_z} g^{-s_z} g_0^{s_\beta c}$. By setting $c_z = H_z(R_z, C_{\text{pk}})$, π_z is zero-knowledge. $\square$

The simulator $\mathcal{S}$ runs $\text{param} \leftarrow \text{Setup}(1^\lambda)$ and $\text{param}_I \leftarrow \text{IS.Setup}(\text{param})$. $\mathcal{S}$ runs $(\text{sk}_i, \text{pk}_i) \leftarrow \text{KeyGen}()$ using randomness ω_i for $i \in [1, n]$ and sets $\text{pk} = (\text{pk}_1, \dots, \text{pk}_n)$. $\mathcal{S}$ sends param , param_I and pk to the adversary $\mathcal{A}$.

The adversary returns a message m^* , two indices i_0, i_1 and a set of public key pk^* .

$\mathcal{S}$ picks a random bit b and random group elements $R, C_{\text{pk}} \in \mathbb{G}$. Since the Pedersen commitment is perfectly hiding, C_{pk} is indistinguishable from a real commitment of pk_{i_b} . By the above lemmas, $\mathcal{S}$ can generate π_z and π'_b that have no information about $z, \vec{b}$ and j , and hence no information about i_b . Hence no PPT adversary

can guess $b^* = b$ with probability larger than $1/2$. The incognito Schnorr signature is anonymous against full key exposure. $\square$

THEOREM 4.4. *The incognito Schnorr signature is unforgeable with respect to insider corruption if the discrete logarithm relation assumption holds in the random oracle model.*

PROOF. The proof relies on a number of lemmas.

LEMMA 4.5. π'_b is computational witness extended emulation for the witness $\vec{b}$.

LEMMA 4.6. π_z is computational witness extended emulation for the witness z such that (R, z) is a valid Schnorr signature for the public key committed in C_{pk} .

PROOF. By lemma 4.5, $\vec{b}$ is a binary vector with $b_j = 1$ for only one $j \in [1, n]$ and C_{pk} is the commitment of pk_j .

Given two distinct challenges $c_{z,1}, c_{z,2}$, we have two transcripts $(R_z, s_{z,1}, s_{\beta,1})$ and $(R_z, s_{z,2}, s_{\beta,2})$. Hence we have:

$$g^{s_{z,1}-s_{z,2}} g_0^{(s_{\beta,1}-s_{\beta,2})c} = R^{c_{z,1}-c_{z,2}} C_{pk}^{c(c_{z,1}-c_{z,2})}$$

$$g^{s_{z,1}-s_{z,2}} g_0^{(s_{\beta,1}-s_{\beta,2})c} = (R(g_0^{\beta} pk_j)^c)^{c_{z,1}-c_{z,2}}.$$

Since g_0 is the hash of c, pk and c is the hash of R , hence we have $g_0^{s_{\beta,1}-s_{\beta,2}} = g_0^{\beta(c_{z,1}-c_{z,2})}$ and:

$$g^{s_{z,1}-s_{z,2}} = (R \cdot pk_j^c)^{c_{z,1}-c_{z,2}}.$$

Hence, we can extract $z = \frac{s_{z,1}-s_{z,2}}{c_{z,1}-c_{z,2}}$ such that $g^z = R \cdot pk_j^c$ and $c = H(m, R)$. $\square$

Setup. The simulator $\mathcal{S}$ acts as attacker to break the unforgeability of the standard signature scheme. $\mathcal{S}$ receives param and pk^* from its challenger. The simulator $\mathcal{S}$ runs $param_I \leftarrow \text{IS.Setup}(param)$. $\mathcal{S}$ picks a random index $j \in [1, n]$. $\mathcal{S}$ runs $(sk_i, pk_i) \leftarrow \text{KeyGen}()$ for $i \in [1, n], i \neq j$. $\mathcal{S}$ sets $pk_j = pk^*$ and $pk = (pk_1, \dots, pk_n)$. $\mathcal{S}$ sends param, $param_I, pk$ to the adversary $\mathcal{A}$.

Oracle Simulation. When $\mathcal{A}$ asks the signing oracle for pk_j and a message m , $\mathcal{S}$ forwards this query to its challenger to get the answer. If $\mathcal{A}$ asks the signing oracle for $pk_i \neq pk_j$, $\mathcal{S}$ computes it using sk_i .

When $\mathcal{A}$ asks the corruption oracle for $pk_i \neq pk_j$, $\mathcal{S}$ returns sk_i . If $\mathcal{A}$ asks the corruption oracle for pk_j , $\mathcal{S}$ declares failure and exits.

When $\mathcal{A}$ asks the conversion oracle for pk_i and a message m and a set of public key pk' , $\mathcal{S}$ will return the incognito signature σ_I calculated from sk_i if $pk_i \neq pk^*$. If $pk_i = pk^*$, then $\mathcal{S}$ honestly computes a commitment C_{pk} . $\mathcal{S}$ picks a random $R \in \mathbb{G}$, $s_z, s_\beta \in \mathbb{Z}_p$ and computes $c = H(m, R)$, $g_0 = H_g(c, pk')$, $R_z = g^{s_z} g_0^{s_\beta c} R^{-c_z} C_{pk}^{-cc_z}$. $\mathcal{S}$ sets $c_z = H_z(R_z, C_{pk})$ in the random oracle H_z and obtains $\pi_z = (R_z, s_z, s_\beta)$. $\mathcal{S}$ honestly computes the proof π'_b . $\mathcal{S}$ returns the incognito signature $\sigma_I = (R, C_{pk}, \pi_z, \pi'_b)$.

Forgery. Finally, $\mathcal{A}$ returns a subset of public keys $pk^* \subseteq pk$, an incognito signature σ_I^* and a message m^* . Denote $\sigma_I^* = (R^*, C_{pk}^*, \pi_z^*, \pi'_b^*)$. By lemma 4.5, $\mathcal{S}$ can extract from π'_b^* a binary vector $\vec{b}$ with the j^* -th entry is equal to 1. If the j^* -th public key pk_{j^*} in pk^* is

not equal to pk^* , $\mathcal{S}$ declares failure and exits. Otherwise, $\mathcal{S}$ can extract some z^* from π_z^* by lemma 4.6, such that

$$g^{z^*} = R^*(pk^*)^{c^*}, \quad c^* = H(m^*, R^*).$$

$\mathcal{S}$ returns (c^*, z^*) to its challenger as the forgery of the Schnorr signature. The only exception is that $\pi_z^* = \pi_z$, which is from the output of a conversion oracle query with input (m, pk', pk^*) , such that $\mathcal{S}$ cannot extract a forgery of the Schnorr signature from this π_z . We show that it is not possible by contradiction. If this $\pi_z = (R_z, s_z, s_\beta)$ is valid for both σ_I and σ_I^* , it means that:

$$R_z g^{-s_z} = g_0^{s_\beta c} R^{-c_z} C_{pk}^{-cc_z} = (g_0^{s_\beta c^*} (R^*)^{-c_z^*} (C_{pk}^*)^{-c^* c_z^*}), \quad (2)$$

where $c = H(m, R)$, $c^* = H(m^*, R^*)$, $c_z = H_z(R_z, C_{pk})$, $c_z^* = H_z(R_z, C_{pk}^*)$, $g_0 = H_g(c, pk')$, $g_0^* = H_g(c^*, pk^*)$. As required by the winning condition, either $m \neq m^*$ or $pk' \neq pk^*$. For either case, we have:

$$g_0 = H_g(H(m, R), pk') \neq H_g(H(m^*, R^*), pk^*) = g_0^*.$$

From Eq. 2, we can see that there is no g_0^* on the left hand side. It means that $(R^*(C_{pk}^*)^{c^*})^{-c_z^*}$ should contains a term $(g_0^*)^{-s_\beta c^*}$ to cancel out the g_0^* term on the right hand side. However, g_0^* is computed from the hash of R^* and hence R^* cannot have any g_0^* term. It remains $(C_{pk}^*)^{-c^* c_z^*}$ that may contains the term $(g_0^*)^{-s_\beta c^*}$. However, c_z^* is calculated from the hash of C_{pk}^* , which makes it impossible for $(C_{pk}^*)^{c_z^*}$ to have a term $(g_0^*)^{s_\beta}$ except with negligible probability. Hence, we reach a contradiction.

Hence if there exists a PPT adversary breaks the unforgeability of the incognito signature, then $\mathcal{S}$ can break the unforgeability of the Schnorr signature, which breaks the discrete logarithm assumption. $\square$

4.2 DSA-based: Incognito (EC)DSA Signature

We review the standard (EC)DSA signature. We adopt the GenDSA framework in [18]. ECDSA is included in standards like ISO/IEC 14888-3, FIPS 186-4 and ANSI X9.62.

Setup. On input a security parameter 1^λ , it outputs a cyclic group $\mathbb{G}$ of prime order p , a generator $g \in \mathbb{G}$ and a hash function $H : \{0, 1\}^* \rightarrow \mathbb{Z}_p$. It also defines a function $f : \mathbb{G} \rightarrow \mathbb{Z}_p^1$.

KeyGen. It picks a random secret key $sk = x$ and computes a public key $pk = g^x$.

Sign. To sign a message m with the secret key x , it picks a random $k \in \mathbb{Z}_p$ and computes $r = f(g^k)$ and $s = k^{-1}(H(m) + rx)$. It outputs the signature $\sigma = (r, s)$.

Verify. To verify a signature σ with a message m and a public key pk , it computes $K = (g^{H(m)} (g^x)^r)^{s^{-1}}$. It outputs 1 if $r = f(K)$ or 0 otherwise.

Incognito (EC)DSA Signature. Observe that the verification of the (EC)DSA signature involves a computation of $r = f(K = (g^{H(m)} (g^x)^r)^{s^{-1}})$. Following the generic construction, we have to use a zk-SNARK for the pre-image of the f function. In order to avoid the complexity of such zk-SNARK, we give a different concrete instantiation with posterior anonymity with respect to

¹For ECDSA, the function $f(g)$ returns the x-coordinates of the point $g \bmod p$.

non-signer (instead of strong posterior anonymity). In particular, the value K is included in the incognito (EC)DSA signature. The concrete construction is as follows.

IS.Setup. It is the same as the incognito Schnorr signature.

IS.Convert. On input a signature $\sigma = (r, s)$, a message m , a public key pk^* , and a set of public keys $\text{pk} = \{\text{pk}_1, \dots, \text{pk}_n\}$, with $\text{pk}_j = \text{pk}^*$ for some index $j \in [1, n]$, it first computes $K = (g^{H(m)} \text{pk}_j^r)^{s^{-1}}$. It converts the verification to:

$$K^s = g^{H(m)} \left(\prod_{i=1}^n \text{pk}_i^{b_i} \right)^r,$$

where $b_j = 1$ and $b_i = 0$ for all $i \neq j$. In order to maintain anonymity, it needs a zero-knowledge proof for all b_i , as well as s . Hence, it runs as follows.

- (1) Commit pk_j . Compute a group element $g_0 = H_g(K, m, \text{pk})$. Pick a random $\beta \in \mathbb{Z}_p$ and compute a commitment $C_{\text{pk}} = g_0^\beta \text{pk}_j = g_0^\beta \text{pk}^{\vec{b}}$, where $\vec{b} = (b_1, \dots, b_n)$.
- (2) ZK Proof for s . Pick random $r_z, r_\beta \in \mathbb{Z}_p$ and compute $R_z = K^{r_z} g_0^{r_\beta r}$, $c_z = H_z(R_z, C_{\text{pk}})$, $s_z = r_z + c_z s$, $s_\beta = r_\beta + c_z \beta$. Denote $\pi_z = (R_z, s_z, s_\beta)$.
- (3) ZK Proof for $\vec{b}$. It is the same as the incognito Schnorr signature for π'_b .

Finally, it outputs the incognito signature $\sigma_I = (K, C_{\text{pk}}, \pi_z, \pi'_b)$.

IS.CVerify. On input an incognito signature $\sigma_I = (K, C_{\text{pk}}, \pi_z, \pi'_b)$, a message m and a set of public key pk , it outputs 1 if all of the following checks hold and outputs 0 otherwise.

For $\pi_z = (R_z, s_z, s_\beta)$, it computes $g_0 = H_g(K, m, \text{pk})$, $c_z = H_z(R_z, C_{\text{pk}})$ and $r = f(K)$, checks if

$$K^{s_z} g_0^{s_\beta r} = R_z (g^{H(m)} C_{\text{pk}}^r)^{c_z}.$$

The verification of π'_b is the same as that in the incognito Schnorr signature.

THEOREM 4.7. *The incognito (EC)DSA signature is posterior anonymous against full key exposure in the random oracle model.*

PROOF. The proof relies on lemma 4.2 and the following lemma.

LEMMA 4.8. *π_z is perfect honest-verifier zero-knowledge for the witness s .*

PROOF. By picking random $c_z, s_z, s_\beta \in \mathbb{Z}_p$, we can compute $r = f(K)$ and $R_z = K^{s_z} g_0^{s_\beta r} g^{-c_z H(m)} C_{\text{pk}}^{-r c_z}$. By setting $c_z = H_z(R_z, C_{\text{pk}})$, π_z is zero-knowledge. $\square$

The rest of the security proof is similar to the security proofs of incognito Schnorr signatures and hence are omitted. $\square$

THEOREM 4.9. *The incognito (EC)DSA signature is unforgeable with respect to insider corruption if the discrete logarithm relation assumption holds in the random oracle model and (EC)DSA signature is existential unforgeable against chosen message attack.*

PROOF. The proof relies on lemma 4.5 and the following lemma.

LEMMA 4.10. *π_z is computational witness extended emulation for the witness s such that (r, s) is a valid (EC)DSA signature for the public key committed in C_{pk} .*

PROOF. By the proof of lemma 4.5, $\vec{b}$ is a binary vector with $b_j = 1$ for only one $j \in [1, n]$ and C_{pk} is the commitment of pk_j .

Given two distinct challenges $c_{z,1}, c_{z,2}$, we have two transcripts $(R_z, s_{z,1}, s_{\beta,1})$ and $(R_z, s_{z,2}, s_{\beta,2})$. Hence we have:

$$\begin{aligned} K^{s_{z,1} - s_{z,2}} g_0^{(s_{\beta,1} - s_{\beta,2})r} &= g^{(c_{z,1} - c_{z,2})H(m)} C_{\text{pk}}^{r(c_{z,1} - c_{z,2})} \\ &= (g^{H(m)} (g_0^\beta \text{pk}_j)^r)^{c_{z,1} - c_{z,2}}. \end{aligned}$$

Since g_0 is the hash of K, m, pk , we have $g_0^{s_{\beta,1} - s_{\beta,2}} = g_0^{\beta(c_{z,1} - c_{z,2})}$ and:

$$K^{s_{z,1} - s_{z,2}} = (g^{H(m)} \cdot \text{pk}_j^r)^{c_{z,1} - c_{z,2}}.$$

Hence, we can extract $s = \frac{s_{z,1} - s_{z,2}}{c_{z,1} - c_{z,2}}$ such that $K^s = g^{H(m)} \cdot \text{pk}_j^r$ and $r = f(K)$. $\square$

For the description of the simulator $\mathcal{S}$, the Setup part, and the simulation of the signing oracle and corruption oracle are the same as those in the security proof of incognito Schnorr signatures and hence are omitted. We continue from the conversion oracle.

When $\mathcal{A}$ asks the conversion oracle for pk_i and a message m and a set of public key pk' , $\mathcal{S}$ will return the incognito signature σ_I calculated from sk_i if $\text{pk}_i \neq \text{pk}^*$. If $\text{pk}_i = \text{pk}^*$, then $\mathcal{S}$ honestly computes a commitment C_{pk} . $\mathcal{S}$ picks a random $K \in \mathbb{G}$, $s_z, s_\beta \in \mathbb{Z}_p$ and computes $r = f(K)$, $g_0 = H_g(K, m, \text{pk}')$, $R_z = K^{s_z} g_0^{s_\beta r} g^{-c_z H(m)} C_{\text{pk}}^{-r c_z}$. $\mathcal{S}$ sets $c_z = H_z(R_z, C_{\text{pk}})$ in the random oracle H_z and obtains $\pi_z = (R_z, s_z, s_\beta)$. $\mathcal{S}$ honestly computes the proof π'_b . $\mathcal{S}$ returns the incognito signature $\sigma_I = (K, C_{\text{pk}}, \pi_z, \pi'_b)$.

Forgery. Finally, $\mathcal{A}$ returns a subset of public keys $\text{pk}^* \subseteq \text{pk}$, an incognito signature σ_I^* and a message m^* . Denote $\sigma_I^* = (K^*, C_{\text{pk}}^*, \pi_z^*, \pi'_b^*)$. By lemma 4.5, $\mathcal{S}$ can extract from π'_b^* a binary vector $\vec{b}$ with the j^* -th entry is equal to 1. If the j^* -th public key pk_{j^*} in pk^* is not equal to pk^* , $\mathcal{S}$ declares failure and exits. Otherwise, $\mathcal{S}$ can extract some s^* from π_z^* by lemma 4.6, such that

$$(K^*)^{s^*} = g^{H(m^*)} (\text{pk}^*)^{r^*}, \quad r^* = f(K^*).$$

$\mathcal{S}$ returns (s^*, r^*) to its challenger as the forgery of the (EC)DSA signature. The only exception is that $\pi_z^* = \pi_z$, which is from the output of a conversion oracle query with input $(m, \text{pk}', \text{pk}^*)$, such that $\mathcal{S}$ cannot extract a forgery of the (EC)DSA signature from this π_z . We show that it is not possible by contradiction. If this $\pi_z = (R_z, s_z, s_\beta)$ is valid for both σ_I and σ_I^* , it means that:

$$\begin{aligned} R_z &= K^{s_z} g_0^{s_\beta r} g^{-c_z H(m)} C_{\text{pk}}^{-r c_z} \\ &= (K^*)^{s_z} (g_0^*)^{s_\beta r^*} g^{-c_z^* H(m^*)} (C_{\text{pk}}^*)^{-r^* c_z^*}, \end{aligned} \quad (3)$$

where $r = f(K)$, $r^* = f(K^*)$, $c_z = H_z(R_z, C_{\text{pk}})$, $c_z^* = H_z(R_z, C_{\text{pk}}^*)$, $g_0 = H_g(K, m, \text{pk}')$, $g_0^* = H_g(K^*, m^*, \text{pk}^*)$. As required by the winning condition, either $m \neq m^*$ or $\text{pk}' \neq \text{pk}^*$. For either case, we have:

$$g_0 = H_g(K, m, \text{pk}') \neq H_g(K^*, m^*, \text{pk}^*) = g_0^*.$$

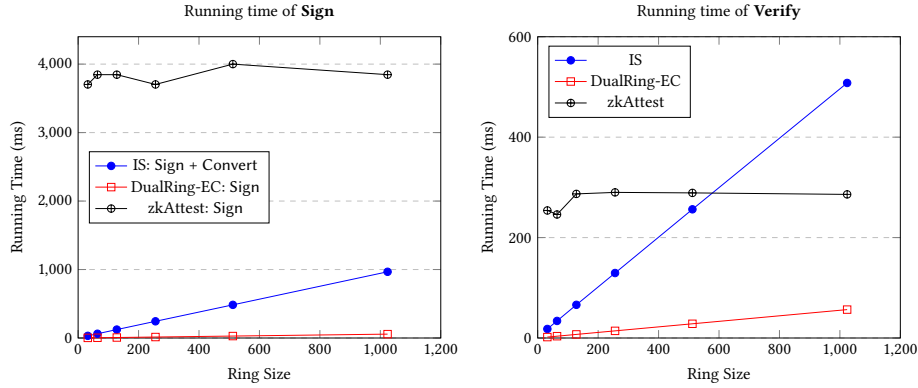


Figure 1: Running times of Incognito Schnorr/ECDSA Signature (IS), zkAttest and DualRing-EC signatures.

Table 1: Comparing with $O(\log n)$ -size DL-based ring signature schemes, for n public keys, where p is a 256-bit prime.

Schemes	# elements in signature		Signature Size (Bytes)				
	$\mathbb{G}$	$\mathbb{Z}_p$	$n = 2$	$n = 8$	$n = 64$	$n = 1024$	$n = 4096$
Ringct 3.0 [35]	$2 \log n + 7$	7	521	653	851	1115	1247
Omniring [25]	$2 \log(n + 2) + 4$	5	424	523	721	1018	1117
DualRing-EC [34]	$2 \log n + 1$	3	195	327	525	789	921
Incognito Schnorr	$2 \log n + 8$	8	586	718	916	1180	1312
zkAttest [17]	$4 \log n + 2804$	$3 \log n + 3361$	197504	197952	198624	199520	199968
Incognito ECDSA	$2 \log n + 8$	8	586	718	916	1180	1312

From Eq. 3, we can see that there is no g_0^* on the left hand side. It means that $(K^*)^{s_z} (C_{pk}^*)^{-r^* c_z^*}$ should contains a term $(g_0^*)^{-s\beta r^*}$ to cancel out the g_0^* term on the right hand side. However, g_0^* is computed from the hash of K^* and hence K^* cannot have any g_0^* term. It remains $(C_{pk}^*)^{-r^* c_z^*}$ that may contains the term $(g_0^*)^{-s\beta r^*}$. However, c_z^* is calculated from the hash of C_{pk}^* , which makes it impossible for $(C_{pk}^*)^{c_z^*}$ to have a term $(g_0^*)^{s\beta}$ except with negligible probability. Hence, we reach a contradiction.

Hence if there exists a PPT adversary breaks the unforgeability of the incognito signature, then $\mathcal{S}$ can break the unforgeability of the (EC)DSA signature. $\square$

Extension to Generic ElGamal Signatures. [18] proposed a more general generic ElGamal signatures, which cover the standardized signature schemes DSA, ECDSA, SM2, GOST, and ECGOST. Our technique can also be applied to the generic ElGamal signatures.

The pairing-based, RSA-based and lattice-based instantiations are given in the appendix.

4.3 Analysis of Incognito Schnorr/ECDSA Signature

Efficiency. For the Incognito Schnorr/ECDSA Signature, it has two group elements for (R, C_{pk}) , π_z has one group element and two elements in $\mathbb{Z}_p$, π_b has $2 \log n + 5$ group element and 6 elements in $\mathbb{Z}_p$. It has a total of $(2 \log n + 8)$ elements in $\mathbb{G}$ and 8 elements in $\mathbb{Z}_p$.

Comparing with the state-of-the-art ring signature schemes in Table 1, our signature size is also $O(\log n)$, but with a larger constant factor.

Compare with the ECDSA-based zkAttest [17], our incognito ECDSA signature is significantly shorter due to the huge constant overhead in zkAttest². zkAttest is at least 152 times longer than the incognito ECDSA signature for $n \leq 4096$.

Implementation. We implement our Incognito Schnorr Signature in Rust, using the secp256k1 curve in the *k256* library³. It is tested on a server with dual Intel Xeon Gold 6226R CPU @ 2.90GHz CPU, 256GB RAM in Ubuntu 20.04. As no prior primitive can provide the desired posterior anonymity feature offered by the proposed Incognito Signatures, we compare its performance with DualRing-EC, a state-of-art privacy-preserving signature, and zkAttest, the only privacy-preserving signature for ECDSA keys in the literature. The comparison, illustrated in Figure 1, provides insight into the relative performance of our proposed method.

We observe that our Incognito Schnorr Signature is slower than DualRing-EC due to the additional features it provides, which DualRing-EC lacks. However, the running time of our Sign algorithm (creating a Schnorr signature) remains highly efficient, taking less than 0.2ms. The majority of the computation time is consumed by the Convert algorithm, yet it still completes in under 1s for a set size of 1024. This makes our method particularly suitable for

²Signature size is not given in the paper [17]. Data is collected from the implementation at <https://github.com/cloudflare/zkp-ecdsa>. They repeat a NIZKAoK for 80 times to achieve the soundness of 2^{-80} .

³<https://github.com/incognito-sig/incognito>.

applications that require a fast signing and can leverage a powerful server for conversion and verification.

In terms of ECDSA, the running time of zkAttest is dominated by the constant overhead of their NIZKAoK on elliptic curve conversion. The running time of the incognito ECDSA signature is almost the same as the incognito Schnorr signature, since it is dominated by the same NIZKAoK. Our incognito ECDSA signature is much faster than zkAttest in signing (74% less running time) for ring size less than 1024. The verification of the incognito ECDSA signature is slower than zkAttest for ring size larger than 512.

5 POSTERIOR MESSAGE HIDING: CONCEALED SIGNATURES

Considering posterior message hiding, we assume that the converter wants to show that the signed message satisfies certain relation (e.g., the signed transaction amount is within a suitable range for cryptocurrency transactions), and yet the message (e.g., the transaction amount) can still be hidden from the verifier. In this section, we define *posterior message hiding* as that the converted signature can be verified and decommitted to the message. The decommitted message can be further used in other zero-knowledge proofs for any NP relation. In this paper, we call this signature as a *concealed signature*.

5.1 Model of Concealed Signatures

We first review a secure commitment scheme

- **Setup_{com}**. On input the security parameter 1^λ , it outputs the public parameters param . The public parameters are ignored from all algorithm's input for simplicity.
- **Com**. On input a message m , it outputs a commitment C and some auxiliary information aux .
- **Decom**. On input a message m , a commitment C and some auxiliary information aux , it outputs 1 or 0.

We define the notion of a concealed signature as a standard signature (Setup , KeyGen , Sign , Verify) with the following algorithms.

- **CS.Setup**. On input the public parameters param of the standard signature, it outputs the concealed signatures' public parameters param_{CS} .
- **CS.Convert**. On input a standard signature σ for a message m , a public key pk , it (1) outputs $\perp$ if $\text{Verify}(\sigma, m, \text{pk}) = 0$ or (2) outputs a concealed signature σ_{CS} and some auxiliary information aux .
- **CS.CVerify**. On input a concealed signature σ_{CS} and a public key pk , it output 1 for valid or 0 for invalid.
- **CS.Decom**. On input a message m , a concealed signature σ_{CS} and some auxiliary information aux , it outputs 1 or 0.

Correctness of the concealed signature means that for all messages m in the message space, $(\text{sk}, \text{pk}) \leftarrow \text{KeyGen}()$, $\sigma \leftarrow \text{Sign}(m, \text{sk})$, $(\sigma_{\text{CS}}, \text{aux}) \leftarrow \text{CS.Convert}(\sigma, m, \text{pk})$, we have:

$$\text{CS.CVerify}(\sigma_{\text{CS}}, \text{pk}) = 1 \wedge \text{CS.Decom}(m, \sigma_{\text{CS}}, \text{aux}) = 1.$$

Unforgeability. We model the case that when given a public key pk , the adversary can query the corresponding signing oracle on any message m , but he cannot create a concealed signature on m^* and pk , with $m^* \neq m$.

Unforgeability of concealed signatures is formally defined in the following game between a challenger $\mathcal{C}$ and an adversary $\mathcal{A}$.

- (1) The challenger $\mathcal{C}$ runs $\text{param} \leftarrow \text{Setup}(1^\lambda)$, and $\text{param}_{\text{CS}} \leftarrow \text{CS.Setup}(\text{param})$. $\mathcal{C}$ runs $(\text{sk}, \text{pk}) \leftarrow \text{KeyGen}()$. $\mathcal{C}$ sends $(\text{param}, \text{param}_{\text{CS}}, \text{pk})$ to the adversary $\mathcal{A}$.
- (2) $\mathcal{A}$ can make adaptive queries to the signing oracle: on input a message m , it returns a signature $\sigma \leftarrow \text{Sign}(m, \text{sk})$.
- (3) $\mathcal{A}$ returns a concealed signature σ_{CS}^* , an auxiliary information aux^* and a message m^* to $\mathcal{C}$.

$\mathcal{A}$ wins the game if $\text{CS.CVerify}(\sigma_{\text{CS}}^*, \text{pk}) = 1$, $\text{CS.Decom}(m^*, \sigma_{\text{CS}}^*, \text{aux}^*) = 1$ and there is no signing oracle query with input m^* . We say that a concealed signature is *unforgeable* if no PPT adversary can win the game with non-negligible probability.

Posterior Message Hiding. We define a *strong posterior message hiding*, so that the signed message remains hidden even if the signing keys and the randomness used in the standard signatures are known to the adversary. It is formally defined in the following game between a challenger $\mathcal{C}$ and an adversary $\mathcal{A}$.

- (1) The challenger $\mathcal{C}$ runs $\text{param} \leftarrow \text{Setup}(1^\lambda)$, and $\text{param}_{\text{CS}} \leftarrow \text{CS.Setup}(\text{param})$. $\mathcal{C}$ runs $(\text{sk}, \text{pk}) \leftarrow \text{KeyGen}()$. $\mathcal{C}$ sends param , param_{CS} , pk , and sk to the adversary $\mathcal{A}$.
- (2) $\mathcal{A}$ returns two messages m_0^*, m_1^* and two signatures $\sigma_0^* \leftarrow \text{Sign}(m_0^*, \text{sk})$, $\sigma_1^* \leftarrow \text{Sign}(m_1^*, \text{sk})$ to $\mathcal{C}$.
- (3) $\mathcal{C}$ picks a random bit b and runs $(\sigma_{\text{CS}}^*, \text{aux}^*) \leftarrow \text{CS.Convert}(\sigma_b^*, m_b^*, \text{pk})$. $\mathcal{C}$ returns σ_{CS}^* to $\mathcal{A}$.
- (4) $\mathcal{A}$ outputs a bit b^* .

If $b^* = b$, then $\mathcal{A}$ wins the game. We say that a concealed signature is *strongly posterior message hiding against full key and signature exposure* if no probabilistic polynomial time (PPT) adversary can win the game with non-negligible probability over $1/2$.

In the model, we allow $\mathcal{A}$ to calculate σ_b^* for $b = 0/1$ in step 2. In step 3, $\mathcal{C}$ converts one of the σ_b^* into σ_{CS}^* . Hence the strong posterior message hiding guarantees that the concealed signature σ_{CS}^* remains message hiding even when the original signing randomness and the signing key are known to the adversary.

5.2 Generic Construction of Concealed Signatures

Given a standard signature scheme (Setup , KeyGen , Sign , Verify), a commitment scheme ($\text{Setup}_{\text{com}}$, Com , Decom) and a non-interactive zero-knowledge argument of knowledge $(\mathcal{K}, \mathcal{P}, \mathcal{V})$, we give a generic construction of concealed signatures.

CS.Setup. The system parameters param_{CS} includes the parameters $\tau \leftarrow \mathcal{K}(1^\lambda)$ and $\text{param}_{\text{com}} \leftarrow \text{Setup}_{\text{com}}(1^\lambda)$.

CS.Convert. On input a signature σ , a message m , a public key pk , it computes $(C, \text{aux}) \leftarrow \text{Com}(m)$ and generates a NIZKAoK $\pi \leftarrow \mathcal{P}(\tau, u, w)$ for the following relation:

$$\{u := (\text{pk}, C); w := (\sigma, m, \text{aux})\}$$

$$\text{Verify}(\sigma, m, \text{pk}) = 1 \wedge \text{Decom}(m, C, \text{aux}) = 1\}.$$

It outputs the concealed signature $\sigma_{\text{CS}} = (\pi, C)$.

CS.CVerify. On input a concealed signature $\sigma_{\text{CS}} = (\pi, C)$ and a public key pk , it outputs 1 if $1 \leftarrow \mathcal{V}(\tau, u := (\text{pk}, C), \pi)$ and outputs 0 otherwise.

CS.Decom. On input a message m , a concealed signature $\sigma_{CS} = (\pi, C)$ and aux, it outputs $\text{Decom}(m, C, \text{aux})$.

THEOREM 5.1. *The generic concealed signature is unforgeable if the underlying signature scheme is unforgeable, the commitment scheme is binding and the NIZKAoK has witness extended emulation.*

PROOF. Setup. The simulator $\mathcal{S}$ receives the system parameters param and a public key pk^* from the challenger $\mathcal{C}$ of the standard signature. The simulator $\mathcal{S}$ receives the system parameters $\text{param}_{\text{com}}$ from the challenger $\mathcal{C}'$ of the commitment scheme. $\mathcal{S}$ generates param_{CS} using the parameters $\text{param}_{\text{com}}$ and $\tau \leftarrow \mathcal{K}(1^\lambda)$. $\mathcal{S}$ generates a key pair (sk, pk) using $\text{KeyGen}()$. $\mathcal{S}$ sends $\text{param}, \text{param}_{CS}, \text{sk}$ and pk to the adversary $\mathcal{A}$ attacking the generic concealed signature.

Oracle Simulation. When $\mathcal{A}$ asks the signing oracle for a message m , $\mathcal{S}$ forwards the query to its challenger $\mathcal{C}$ to answer.

Forgery. Finally $\mathcal{A}$ returns a concealed signature $\sigma_{CS}^* = (\pi^*, C^*)$, an auxiliary information aux^* and a message m^* , with $\text{CS.CVerify}(\sigma_{CS}^*, \text{pk}) = 1$ and $\text{Decom}(m^*, C^*, \text{aux}^*) = 1$. There was no signing oracle query with input m^* . By the witness extended emulation property of the NIZKAoK, $\mathcal{S}$ can extract $(\sigma', m', \text{aux}')$ so that:

$$\text{Verify}(\sigma', m', \text{pk}) = 1, \quad \text{Decom}(m', C^*, \text{aux}') = 1.$$

If $m' = m^*$, $\mathcal{S}$ returns (σ', m') as the forgery of the underlying signature scheme. If $m' \neq m^*$, it means that C^* can be decommitted to both m^* and m' , which breaks the binding property of the commitment scheme.

If a PPT adversary can break the unforgeability of the concealed signature with non-negligible probability, $\mathcal{S}$ can also break the unforgeability of the underlying signature in polynomial time with non-negligible probability. Hence, we reach a contradiction. $\square$

THEOREM 5.2. *The generic concealed signature is strongly posterior message hiding against full key and signature exposure if the NIZKAoK has zero-knowledge property and the commitment scheme is hiding.*

PROOF. By the zero-knowledge property of the NIZKAoK, π does not leak the information of σ , m and aux . By the hiding property of the commitment scheme, C does not leak the information of m . $\square$

5.3 Instantiation of Concealed Signature

If we want to apply the generic concealed signature to a standard signature which computes a hash of the message, we require a ZK proof that support the proof of the pre-image of a hash function, such as zk-SNARK. Alternatively, structure-preserving signatures [23] can be used with the improved Groth-Sahai proof [16] to give an efficient instantiation. In this section, we demonstrate another efficient instantiation from the Boneh-Boyen signature [7], which is reviewed below.

Setup. On input a security parameter 1^λ , it outputs cyclic groups $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$ of prime order p , generators $g_1 \in \mathbb{G}_1, g_2 \in \mathbb{G}_2$ and a hash function $H : \{0, 1\}^* \rightarrow \mathbb{G}_1$. It also defines a pairing $\hat{e} : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$.

KeyGen. It picks a random secret key $\text{sk} = (x, y)$ and computes a public key $\text{pk} = (X = g_2^x, Y = g_2^y)$.

Sign. To sign a message m with the secret key x , it picks a random $r \in \mathbb{Z}_p$ and computes the signature $\sigma = (s = g_1^{1/x+m+yr}, r)$.

Verify. To verify a signature $\sigma = (s, r)$ with a message m and a public key $\text{pk} = (X, Y)$, it outputs 1 if $\hat{e}(s, X \cdot g_2^m \cdot Y^r) = \hat{e}(g_1, g_2)$ or 0 otherwise.

Concealed Boneh-Boyen Signature. The concealed Boneh-Boyen signature is shown below.

CS.Setup. The system parameters param_{CS} includes a random generators h_2 from $\mathbb{G}_2$ and parameters for the improved Groth-Sahai proof [16].

CS.Convert. On input a signature $\sigma = (s, r)$, a message m , a public key $\text{pk} = (X, Y)$, it picks a random $t \in \mathbb{Z}_p$ and computes $C = g_2^m h_2^t$. It converts the verification equation to:

$$\hat{e}(s, X \cdot C \cdot h_2^{-t} \cdot Y^r) = \hat{e}(g_1, g_2).$$

In order to maintain message hiding, it needs a zero-knowledge proof for t , as well as (s, r) . It generates the improved Groth-Sahai proof [16] π_{GS} for pairing product equation $\hat{e}(s, XC) \cdot \hat{e}(s, h_2^{-t} Y^r) = \hat{e}(g_1, g_2)$ for witness (s, r, t) . Finally, it outputs the concealed signature $\sigma_{CS} = (\pi_{GS}, C)$ and $\text{aux} = t$.

CS.CVerify. On input a concealed signature $\sigma_{CS} = (\pi_{GS}, C)$ and a public key pk , it outputs 1 if π_{GS} is valid and outputs 0 otherwise.

CS.Decom. On input a message m , a concealed signature $\sigma_{CS} = (\pi_{GS}, C)$ and $\text{aux} = t$, it outputs 1 if $C = g_2^m h_2^t$ and outputs 0 otherwise.

THEOREM 5.3. *The concealed Boneh-Boyen signature is unforgeable if the q -SDH assumption holds.*

PROOF. The Boneh-Boyen signature is unforgeable if the q -SDH assumption holds, the Pedersen commitment scheme is binding if the DL assumption holds, and the Groth-Sahai proof has perfect soundness. The q -SDH assumption implies the DL assumption. $\square$

THEOREM 5.4. *The concealed Boneh-Boyen signature is strongly posterior message hiding against full key and signature exposure if the SXDH assumption holds.*

PROOF. The Groth-Sahai proof is zero-knowledge if the SXDH assumption holds. The Pedersen commitment scheme is perfectly hiding. $\square$

6 INCOGNITO CONCEALED SIGNATURE

This paper introduced posterior anonymity and posterior message hiding. One can compose the notion of *incognito concealed signature* that satisfy both properties simultaneously. The security model of concealed incognito signature includes posterior anonymity, posterior message hiding and unforgeability. We consider the case that the converter hides the public key and the message at the same time⁴.

⁴Theoretically, we can also define variants that one converter hides the public key first and then another converter hides the message, or vice versa. We do not consider that in this paper for simplicity.

As a result, we can obtain a generic concealed incognito signature, by using a NIZKAoK for the relation:

$$\{(C, \mathbf{pk}; \sigma, \mathbf{pk}, \vec{b}, m, \text{aux}) : \text{Verify}(\sigma, m, \mathbf{pk}) = 1 \wedge \mathbf{pk} = \prod_{i=1}^n \mathbf{pk}_i^{b_i} \wedge \vec{b} \in \{0, 1\}^n \wedge \|\vec{b}\|_1 = 1 \wedge \text{Decom}(m, C, \text{aux}) = 1.\}$$

The generic concealed incognito signature can be efficiently instantiated by the structure-preserving signatures [23] or the Boneh-Boyen signature [7].

Incognito Concealed Boneh-Boyen Signature. Given the Boneh-Boyen signature (Setup, KeyGen, Sign, Verify) and the improved Groth-Sahai proof [16] $(\mathcal{K}, \mathcal{P}, \mathcal{V})$, we give the construction of incognito concealed Boneh-Boyen signature.

ICS.Setup. The system parameters $\text{param}_{\text{ICS}}$ includes a random generators h_2 from $\mathbb{G}_2$ and $\tau \leftarrow \mathcal{K}(1^\lambda)$.

ICS.Convert. On input a signature $\sigma = (s, r)$, a message m , a public key $\mathbf{pk} = (X^*, Y^*)$, and a set of public keys $\{X_i, Y_i\}_{i \in [n]}$ (with $X^* = X_j, Y^* = Y_j$), it picks a random $t \in \mathbb{Z}_p$ and computes $C = g_2^m h_2^t$. By setting $b_j = 1$ and $b_i = 0$ for $i \in [1, n], i \neq j$, it converts the verification equation to:

$$\hat{e}(s, \prod_{i=1}^n X_i^{b_i} \cdot C \cdot h_2^{-t} \cdot (\prod_{i=1}^n Y_i^{b_i})^r) = \hat{e}(g_1, g_2).$$

In order to maintain message hiding, it needs a NIZKAoK for $t, s, r, b_1, \dots, b_n$. It generates the improved Groth-Sahai proof [16] π_{GS} for pairing product equation $\hat{e}(s, X^* Y^{*r}) \cdot \hat{e}(s, C) \cdot \hat{e}(s, h_2^{-t}) = \hat{e}(g_1, g_2)$ using witness (s, t, r, X^*, Y^*) . It also generates the proof for the relation $X^* = \prod_{i=1}^n X_i^{b_i}, Y^* = \prod_{i=1}^n Y_i^{b_i}, \vec{b} \in \{0, 1\}^n \wedge \|\vec{b}\|_1 = 1$ using witness $(b_1, \dots, b_n, X^*, Y^*)$. Finally, it outputs the concealed signature $\sigma_{\text{ICS}} = (\pi_{\text{GS}}, C)$ and $\text{aux} = t$.

ICS.CVerify. On input an incognito concealed signature $\sigma_{\text{ICS}} = (\pi_{\text{GS}}, C)$ and a set of public key $\mathbf{pk}$, it outputs 1 if π_{GS} is valid and outputs 0 otherwise.

ICS.Decom. On input a message m , an incognito concealed signature $\sigma_{\text{ICS}} = (\pi_{\text{GS}}, C)$ and $\text{aux} = t$, it outputs 1 if $C = g_2^m h_2^t$ and outputs 0 otherwise.

Application. One example of the application of incognito concealed signature is privacy-preserving central bank digital currency (CBDC). We consider the two-tier CBDC architecture [3], in which the commercial banks collect signed transactions from their customers and submit them to the central bank's ledger. In order to protect the transaction privacy, the commercial banks can convert the standard signatures created from their customers' wallets into incognito concealed signatures. The public key is hidden to protect the signer's identity. The message is committed to hide the transaction amount. When the commercial bank wants to prove that the transaction amount falls within a valid range $[\min, \max]$, it can prove in zero-knowledge for the witness m and aux so that:

$$1 = \text{ICS.Decom}(m, \sigma_{\text{ICS}}, \text{aux}) \wedge m \in [\min, \max].$$

As a result, the two-tier CBDC can achieve the level of privacy similar to Monero in cryptocurrency with respect to the verifier. At the same time, the standard signature is still signed by the customer wallet for efficient signature generation.

7 CONCLUSION

In this paper, we introduce the concept of *posterior security*. We propose *incognito signatures*, which allows the conversion from a standard signature to an anonymous signature by a non-signer. Our generic incognito signature can be efficiently applied to Schnorr signatures, ECDSA, Falcon, etc. We propose *concealed signatures*, which allows the conversion from a standard signature to a message hiding signature. Our generic incognito concealed signature can be efficiently applied to the Boneh-Boyen signature.

ACKNOWLEDGMENTS

The authors would like to thank Danny Chau, Artus Cheung, York Tsang from the HKMA and Jack Ho from the BIS for the discussion of the two-tier CBDC. Jiangshan Yu and Joseph K. Liu are funded by the Australian Research Council (DP220101234).

REFERENCES

- [1] Masayuki Abe, Miyako Ohkubo, and Koutarou Suzuki. 2002. 1-out-of-n Signatures from a Variety of Keys. In *ASIACRYPT 2002 (Lecture Notes in Computer Science, Vol. 2501)*, Yuliang Zheng (Ed.). Springer, 415–432.
- [2] Giuseppe Ateniese, Daniel H. Chou, Breno de Medeiros, and Gene Tsudik. 2005. Sanitizable Signatures. In *ESORICS 2005 (Lecture Notes in Computer Science, Vol. 3679)*, Sabrina De Capitani di Vimercati, Paul F. Syverson, and Dieter Gollmann (Eds.). Springer, 159–177.
- [3] Bank for International Settlements. 2022. Project Aurum. A Prototype for Two-tier Central Bank Digital Currency (CBDC). BIS Innovation Hub.
- [4] Adam Bender, Jonathan Katz, and Ruggero Morselli. 2006. Ring Signatures: Stronger Definitions, and Constructions Without Random Oracles. In *TCC 2006 (Lecture Notes in Computer Science, Vol. 3876)*, Shai Halevi and Tal Rabin (Eds.). Springer, 60–79.
- [5] Alexander R. Block, Justin Holmgren, Alon Rosen, Ron D. Rothblum, and Pratik Soni. 2021. Time- and Space-Efficient Arguments from Groups of Unknown Order. In *CRYPTO 2021 (Lecture Notes in Computer Science, Vol. 12828)*, Tal Malkin and Chris Peikert (Eds.). Springer, 123–152.
- [6] Jan Bobolz, Jesus Diaz, and Markulf Kohlweiss. 2024. Foundations of Anonymous Signatures: Formal Definitions, Simplified Requirements, and a Construction Based on General Assumptions. In *FC 2024 (Lecture Notes in Computer Science, Vol. 14745)*, Jeremy Clark and Elaine Shi (Eds.). Springer, 121–139.
- [7] Dan Boneh and Xavier Boyen. 2008. Short Signatures Without Random Oracles and the SDH Assumption in Bilinear Groups. *J. Cryptol.* 21, 2 (2008), 149–177.
- [8] Dan Boneh, Xavier Boyen, and Hovav Shacham. 2004. Short Group Signatures. In *CRYPTO 2004 (Lecture Notes in Computer Science, Vol. 3152)*, Matthew K. Franklin (Ed.). Springer, 41–55.
- [9] Dan Boneh, Ben Lynn, and Hovav Shacham. 2001. Short Signatures from the Weil Pairing. In *ASIACRYPT 2001 (Lecture Notes in Computer Science, Vol. 2248)*, Colin Boyd (Ed.). Springer, 514–532.
- [10] B. Bünz, J. Bootle, D. Boneh, A. Poelstra, P. Wuille, and G. Maxwell. 2018. Bulletproofs: Short Proofs for Confidential Transactions and More. In *SP 2018*. IEEE Computer Society, 315–334.
- [11] Kwan Yin Chan, Liqun Chen, Yangguang Tian, and Tsz Hon Yuen. 2024. Reconstructing Chameleon Hash: Full Security and the Multi-Party Setting. In *ASIA CCS 2024*, Jianying Zhou, Tony Q. S. Quek, Debin Gao, and Alvaro A. Cárdenas (Eds.). ACM, 1066–1081.
- [12] David Chaum. 1982. Blind Signatures for Untraceable Payments. In *CRYPTO '82*, David Chaum, Ronald L. Rivest, and Alan T. Sherman (Eds.). Plenum Press, New York, 199–203.
- [13] David Chaum. 1985. Security Without Identification: Transaction Systems to Make Big Brother Obsolete. *Commun. ACM* 28, 10 (1985), 1030–1044.
- [14] David Chaum and Eugène van Heyst. 1991. Group Signatures. In *EUROCRYPT '91 (Lecture Notes in Computer Science, Vol. 547)*, Donald W. Davies (Ed.). Springer, 257–265.
- [15] Yvo Desmedt and Yair Frankel. 1989. Threshold Cryptosystems. In *CRYPTO '89 (Lecture Notes in Computer Science, Vol. 435)*, Gilles Brassard (Ed.). Springer, 307–315.
- [16] Alex Escala and Jens Groth. 2014. Fine-Tuning Groth-Sahai Proofs. In *PKC 2014 (Lecture Notes in Computer Science, Vol. 8383)*, Hugo Krawczyk (Ed.). Springer, 630–649.
- [17] Armando Faz-Hernández, Watson Ladd, and Deepak Maram. 2021. ZKAttest: Ring and Group Signatures for Existing ECDSA Keys. In *SAC 2021 (Lecture Notes in Computer Science, Vol. 13203)*, Riham AlTawy and Andreas Hülsing (Eds.). Springer, 68–83.

- [18] Manuel Fersich, Eike Kiltz, and Bertram Poettering. 2016. On the Provable Security of (EC)DSA Signatures. In *CCS 2016*, Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi (Eds.). ACM, 1651–1662.
- [19] Pierre-Alain Fouque, Jeffrey Hoffstein, Paul Kirchner, Vadim Lyubashevsky, Thomas Pornin, Thomas Prest, Thomas Ricosset, Gregor Seiler, William Whyte, and Zhenfei Zhang. 2019. Falcon: Fast-Fourier Lattice-based Compact Signatures over NTRU. <https://falcon-sign.info/falcon.pdf>.
- [20] Rosario Gennaro and Steven Goldfeder. 2018. Fast Multiparty Threshold ECDSA with Fast Trustless Setup. In *CCS 2018*, David Lie, Mohammad Mannan, Michael Backes, and XiaoFeng Wang (Eds.). ACM, 1179–1194.
- [21] Jens Groth and Markulf Kohlweiss. 2015. One-Out-of-Many Proofs: Or How to Leak a Secret and Spend a Coin. In *EUROCRYPT 2015 (Lecture Notes in Computer Science, Vol. 9057)*, Elisabeth Oswald and Marc Fischlin (Eds.). Springer, 253–280.
- [22] Louis C. Guillou and Jean-Jacques Quisquater. 1988. A "Paradoxical" Identity-Based Signature Scheme Resulting from Zero-Knowledge. In *CRYPTO '88 (Lecture Notes in Computer Science, Vol. 403)*, Shafi Goldwasser (Ed.). Springer, 216–231.
- [23] Eike Kiltz, Jiaxin Pan, and Hoeteck Wee. 2015. Structure-Preserving Signatures from Standard Assumptions, Revisited. In *CRYPTO 2015 (Lecture Notes in Computer Science, Vol. 9216)*, Rosario Gennaro and Matthew Robshaw (Eds.). Springer, 275–295.
- [24] Hugo Krawczyk and Tal Rabin. 2000. Chameleon Signatures. In *NDSS 2000*.
- [25] Russell W. F. Lai, Viktoria Ronge, Tim Ruffing, Dominique Schröder, Sri Aravinda Krishnan Thyagarajan, and Jiafan Wang. 2019. Omniring: Scaling Private Payments Without Trusted Setup. In *CCS 2019*. ACM, 31–48.
- [26] Jonathan Lee. 2021. Dory: Efficient, Transparent Arguments for Generalised Inner Products and Polynomial Commitments. In *TCC 2021 (Lecture Notes in Computer Science, Vol. 13043)*, Kobbi Nissim and Brent Waters (Eds.). Springer, 1–34.
- [27] Benoît Libert, Khoa Nguyen, Thomas Peters, and Moti Yung. 2021. Bifurcated Signatures: Folding the Accountability vs. Anonymity Dilemma into a Single Private Signing Scheme. In *EUROCRYPT 2021 (Lecture Notes in Computer Science, Vol. 12698)*, Anne Canteaut and François-Xavier Standaert (Eds.). Springer, 521–552.
- [28] Vadim Lyubashevsky, Ngoc Khanh Nguyen, and Maxime Plançon. 2022. Lattice-Based Zero-Knowledge Proofs and Applications: Shorter, Simpler, and More General. In *CRYPTO 2022 (Lecture Notes in Computer Science, Vol. 13508)*, Yevgeniy Dodis and Thomas Shrimpton (Eds.). Springer, 71–101.
- [29] Khoa Nguyen, Fuchun Guo, Willy Susilo, and Guomin Yang. 2022. Multimodal Private Signatures. In *CRYPTO 2022 (Lecture Notes in Computer Science, Vol. 13508)*, Yevgeniy Dodis and Thomas Shrimpton (Eds.). Springer, 792–822.
- [30] Xianrui Qin, Cailing Cai, and Tsz Hon Yuen. 2021. One-More Unforgeability of Blind ECDSA. In *ESORICS 2021 (Lecture Notes in Computer Science, Vol. 12973)*, Elisa Bertino, Haya Schulmann, and Michael Waidner (Eds.). Springer, 313–331.
- [31] Ronald L. Rivest, Adi Shamir, and Yael Tauman. 2001. How to Leak a Secret. In *ASIACRYPT 2001 (Lecture Notes in Computer Science, Vol. 2248)*, Colin Boyd (Ed.). Springer, 552–565.
- [32] Claus-Peter Schnorr. 1989. Efficient Identification and Signatures for Smart Cards. In *CRYPTO '89 (Lecture Notes in Computer Science, Vol. 435)*, Gilles Brassard (Ed.). Springer, 239–252.
- [33] Guomin Yang, Duncan S. Wong, Xiaotie Deng, and Huaxiong Wang. 2006. Anonymous Signature Schemes. In *PKC 2006 (Lecture Notes in Computer Science, Vol. 3958)*, Moti Yung, Yevgeniy Dodis, Aggelos Kiayias, and Tal Malkin (Eds.). Springer, 347–363.
- [34] Tsz Hon Yuen, Muhammed F. Esgin, Joseph K. Liu, Man Ho Au, and Zhimin Ding. 2021. DualRing: Generic Construction of Ring Signatures with Efficient Instantiations. In *CRYPTO 2021 (Lecture Notes in Computer Science, Vol. 12825)*, Tal Malkin and Chris Peikert (Eds.). Springer, 251–281.
- [35] Tsz Hon Yuen, Shifeng Sun, Joseph K. Liu, Man Ho Au, Muhammed F. Esgin, Qingzhao Zhang, and Dawu Gu. 2020. RingCT 3.0 for Blockchain Confidential Transaction: Shorter Size and Stronger Security. In *FC 2020 (Lecture Notes in Computer Science, Vol. 12059)*, Joseph Bonneau and Nadia Heninger (Eds.). Springer, 464–483.

A RSA-BASED AND LATTICE-BASED INSTANTIATIONS

A.1 RSA-based: Incognito GQ Signature

We now consider the RSA-based Guillou–Quisquater (GQ) signature [22]. We review the GQ signature.

Setup. On input a security parameter 1^λ , it picks two random larger primes p, q and computes $n = pq$. It selects an integer v such that $0 < v < \varphi(n)$ and $\gcd(v, \varphi(n)) = 1$, where $\varphi(n)$ is the Euler function. It

defines a hash function $H : \{0, 1\}^* \rightarrow \mathbb{Z}_{\varphi(n)-1}$. The public parameters are (n, v, H) .

KeyGen. It picks a random secret key $sk = B \in \mathbb{Z}_n$ and computes a public key $pk = J = B^{-v} \bmod n$.

Sign. To sign a message m with the secret key B , it picks a random $r \in \mathbb{Z}_n$ and computes $T = r^v \bmod n$, $h = H(m, T)$ and $t = rB^h \bmod n$. It outputs the signature (t, h) .

Verify. To verify a signature σ with a message m and a public key $pk = J$, it computes $T = t^v J^h \bmod n$. It outputs 1 if $h = H(m, T)$ or 0 otherwise.

Incognito GQ Signature. We give the incognito GQ signature.

IS.Setup. The system parameters param_I includes the parameters for RSA-based commitment and ZK proof.

IS.Convert. On input a signature σ , a message m , a public key pk^* and a set of public keys $\mathbf{pk} = \{pk_1, \dots, pk_n\}$, with $pk_j = pk^*$ for some $j \in [1, n]$, it converts the verification to:

$$T = t^v \left(\prod_{i=1}^n pk_i^{b_i} \right)^h \bmod n,$$

where $b_j = 1$ and $b_i = 0$ for all $i \neq j$. In order to maintain anonymity, it needs a zero-knowledge proof for all b_i , as well as t . There are a few steps to fulfil this goal.

- (1) Commit pk_j . Compute a commitment $C_{pk} = \text{Com}(pk_j) = \text{Com}(\mathbf{pk}^{\vec{b}})$, where $\vec{b} = (b_1, \dots, b_n)$.
- (2) ZK Proof for $t, \vec{b}$. It requires a RSA-based ZK proof π_{RSA} for t and $\vec{b}$, such as the ZK proof for unknown order group in [5].

Finally, it outputs the incognito signature $\sigma_I = (C_{pk}, \pi_{RSA})$.

IS.Verify. On input an incognito signature $\sigma_I = (C_{pk}, \pi_{RSA})$, a message m and a set of public key $\mathbf{pk}$, it outputs 1 if π_{RSA} is valid and outputs 0 otherwise.

The security of the incognito GQ signature follows the security proof of the generic incognito signature.

Extension. Our generic incognito signature can also be applied to signature based on class group. However, our generic incognito signature cannot be applied to signature schemes that each user's signature is verified in different modulus (e.g., RSA signature).

A.2 Lattice-based: Incognito Falcon Signature

Falcon [19] is a post-quantum signature scheme selected by the NIST for standardisation in 2022. We review the Falcon signature.

Setup. On input a security parameter 1^λ , it defines a polynomial ring $R_q := \mathbb{Z}_q[x]/\phi$, where $n = 512$ or 1024 , $\phi = x^n + 1$ and $q = 12289$, and a hash function $H : \{0, 1\}^* \rightarrow R_q$. Denote a standard deviation $\sigma = 1.17\sqrt{q/2n}$ and a norm $\beta = 1.1\sigma\sqrt{2n}$.

KeyGen. It chooses random polynomials f and g in R_q from discrete Gaussian distribution over $\mathbb{Z}$ with of standard deviation $\sigma = 1.17\sqrt{q/2n}$. It finds polynomials F, G in R_q such that $fG - gF = q \bmod \phi$ and sets $\vec{B} := \begin{bmatrix} g & -f \\ G & -F \end{bmatrix}$, so that the Gram-Schmidt norm of $\vec{B}$ is at most $1.17\sqrt{q}$. The public key pk will be set as $h = gf^{-1}$

mod q . The secret key sk can be computed from the Fast Fourier Transform (FFT) of $\vec{B}$ and a Falcon tree [19].

Sign. To sign a message m with the secret key sk , it samples $r \leftarrow \{0, 1\}^{320}$ and computes $c = H(r||m, q, n)$. It uses sk to produce a pair of short polynomials (s_1, s_2) such that $s_1 + hs_2 = c \bmod (\phi, q)$ and $\|(s_1, s_2)\|^2 < \lfloor \beta^2 \rfloor$. It outputs the signature (r, s_2) .

Verify. To verify a signature $\sigma = (r, s_2)$ with a message m and a public key h , it computes $c = H(r||m, q, n)$ and $s_1 = c - hs_2 \bmod (\phi, q)$. It outputs 1 if $\|(s_1, s_2)\|^2 < \lfloor \beta^2 \rfloor$, or outputs 0 otherwise.

Incognito Falcon Signature. We give the construction below.

IS.Setup. The system parameters param_I includes the parameters for lattice-based commitment and ZK proof.

IS.Convert. On input a signature σ , a message m , a public key h^* and a set of public keys $\mathbf{pk} = \{h_1, \dots, h_n\}$, with $h_j = h^*$ for some $j \in [1, n]$. it converts the verification to:

$$s_1 + \left(\sum_{i=1}^n b_i h_i \right) s_2 = c \bmod (\phi, q),$$

where $b_j = 1$ and $b_i = 0$ for all $i \neq j$. In order to maintain anonymity, it requires a lattice-based ZK proof π_{lat} for s_2 and $\vec{b}$, such that $s_1 + (\sum_{i=1}^n b_i h_i) s_2 = c \bmod (\phi, q)$ and $\|s_1\|^2 + \|s_2\|^2 < \lfloor \beta^2 \rfloor$. Finally, it outputs the incognito signature $\sigma_I = (C_{pk}, \pi_{lat}, r, s_1)$.

IS.Verify. On input an incognito signature $\sigma_I = (C_{pk}, \pi_{lat}, r, s_1)$, a message m and a set of public key $\mathbf{pk}$, it outputs 1 if π_{lat} is valid and outputs 0 otherwise.

Construction of ZK. For the construction of zero-knowledge proof, we give a brief description here and refer to [28] for detailed proving schemes. This proving framework grants us a large challenge space and the quadratic form of secrets. As a result, the previous feature takes less repetitions in proving and the latter one give the capability to produce a product of secrets, i.e. $\Gamma = s_2 h$.

We choose the ABDLOP commitment scheme [28] which is consistent with the proving system. This commitment scheme provides hiding and binding for a pair of message $(\mathbf{m}_1, \mathbf{m}_2)$, of which $\mathbf{m}_1 \in \mathcal{R}_p^{m_1}$ should be short enough to fit into MSIS problem and $\mathbf{m}_2 \in \mathcal{R}_p^{m_2}$ could be arbitrarily large in $\mathcal{R}_p := \mathbb{Z}_p[x]/\phi$. Here p is a prime number for proof system $p \equiv 5 \pmod{8}$ and we pick the same ϕ as Falcon. Apparently, the binary vector $\mathbf{b}$ could be put into $\mathbf{m}_1$ part directly, and whether s_2 and other parameter fits into or not depends on the MSIS parameter. However, we argue that the proof system works perfectly even though these parameters doesn't fit $\mathbf{m}_1$ well.

Here, the norms are defined by any vector of polynomials $\mathbf{u} = (u_1, \dots, u_n) \in \mathcal{R}_q^n$ and a single polynomial $v = \sum_{i=0}^{d-1} v_i X^i \in \mathcal{R}_q$,

$$\|\mathbf{u}\|_k = \sqrt[k]{\sum_{i=1}^n \|u_i\|^k}, \quad \|v\|_k = \sqrt[k]{\sum_{i=0}^{d-1} \|v_i\|^k}.$$

Apparently, if we put the binary vector $\vec{b} \in \{0, 1\}^n$ directly into the extension field $\mathbf{b} \in \mathcal{R}_p^n$, proving $\|\mathbf{b}\| = 1$ covers most of our need except that b_i could be not 1 but $b_i = \pm X^j$ for $j \in \{0, \dots, d-1\}$. But by the observation $\|uv\| \leq \|u\|_1 \|v\|$, $\|b_i s_2\| \leq \|s_2\|$ and $\|b_i s_2\|$ is still a valid part of signature.

Now, we need to consider whether $s_2 h$ is able to be computed and proved in the $\mathcal{R}_p$ circumstance. One should have notice that $q \ll p$ in the typical parameters shown by [28], which means that when computing in $\mathcal{R}$ or $\mathcal{R}_p$, the distribution of $s_2 h$ would not have short enough norm. So we pick p that is large enough such that $s_2 h$ coefficients would not go beyond $[-p, p]$. And thus it is able to express $s_2 h \bmod (\phi, q)$ in $\mathcal{R}_p$ as $s_2 h - q\delta$, where $\delta \in \mathcal{R}_p$ is the deviation from $\mathbb{Z}_q$ coefficients to $\mathbb{Z}$.

In this paper, such proving scheme is able to prove (1) the evaluation result $f(\mathbf{s}) = 0$; (2) the constant coefficient of $f(\mathbf{s})$ is 0; (3) $\|\mathbf{Es} - \mathbf{v}\| \leq \beta$; (4) $\mathbf{E}_{\text{bin}} \mathbf{s} - \mathbf{v}_{\text{bin}}$ is indeed binary where

$$f(\mathbf{s}) = \mathbf{s}^\top \mathbf{R}_2 \mathbf{s} + \mathbf{r}_1^\top \mathbf{s} + r_0.$$

For simplicity, we omit the automorphism within the secret⁵. All we need to use as the secret is $\mathbf{s} = (s_2, b_1, b_2, \dots, b_n, \delta)^\top \in \mathcal{R}_p^{n+2}$.

The first part of the ZK proof is to check the l_2 norm of $\mathbf{b}$, which could be checked by automorphism and presented in [28]. Moreover, it should be noted that the evaluation of l_2 norm is under $\mathbb{Z}_p$ setting, which doesn't suffice to prove the vector is indeed binary. So, we need an additional check of this property by the part (4).

The second part of the ZK proof is to check $s_1 + \Gamma = c$. Consequently, the Falcon equation could be expressed into:

$$\mathbf{R} = \begin{bmatrix} 0 & h_1 & h_2 & h_3 & \cdots & h_n & 0 \\ 0 & 0 & 0 & 0 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \cdots & 0 & 0 \end{bmatrix}, \quad \mathbf{r} = \begin{bmatrix} 0 \\ \vdots \\ 0 \\ q \end{bmatrix},$$

$$f(\mathbf{s}) = \mathbf{s}^\top \mathbf{R} \mathbf{s} + \mathbf{r}^\top \mathbf{s} + (-\Gamma).$$

The third part of the ZK proof is to check the (approximate) l_2 bound of s_2 , i.e.

$$\mathbf{E} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 0 \end{bmatrix}, \quad \mathbf{v} = \mathbf{0}, \quad \|\mathbf{Es} + \mathbf{v}\| \leq \beta^2 - \|s_1\|^2.$$

In Falcon, the maximum signature square norm $\lfloor \beta^2 \rfloor$ is 34034726 for Falcon-512 and 70265242 for Falcon-1024. And $\lceil \log_2(\beta) \rceil = 26$. By $\lceil \log_2(\beta) \rceil \leq d$, it would work to prove l_2 norm of s_2 .⁶

B PAIRING-BASED INSTANTIATIONS

B.1 Pairing-based: Incognito BLS Signature

Ethereum 2.0 uses BLS signatures [9] for its proof-of-stake consensus mechanism. We review the BLS signature.

Setup. On input a security parameter 1^λ , it outputs cyclic groups $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$ of prime order p , generators $g_1 \in \mathbb{G}_1, g_2 \in \mathbb{G}_2$ and a hash function $H : \{0, 1\}^* \rightarrow \mathbb{G}_1$. It also defines a pairing $\hat{e} : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$.

KeyGen. It picks a random secret key $sk = x$ and computes a public key $pk = g_2^x$.

⁵The scheme might relies on σ_{-1} automorphism. Check the original paper for details.

⁶Proving $\|\mathbf{s}\| \leq \beta$ is achieved by proving a short binary vector is indeed $\beta^2 - \|s\|^2$.

Sign. To sign a message m with the secret key x , it computes the signature $\sigma = H(m)^x$.

Verify. To verify a signature σ with a message m and a public key pk , it outputs 1 if $\hat{e}(\sigma, g_2) = \hat{e}(H(m), pk)$ or 0 otherwise.

Incognito BLS Signature. We give the incognito BLS signature.

IS.Setup. The system parameters param_I includes random independent generators $h, \vec{g} = (g_1, \dots, g_n), \vec{h} = (h_1, \dots, h_n)$ from $\mathbb{G}_2$, and hash functions $H_g : \mathbb{Z}_p \times \mathbb{G}_2^n \rightarrow \mathbb{G}_2, H_z : \mathbb{G}_2^2 \rightarrow \mathbb{Z}_p, H_w : \mathbb{G}_2^5 \rightarrow \mathbb{Z}_p^2, H_x : \mathbb{G}_2^2 \times \mathbb{Z}_p^2 \rightarrow \mathbb{Z}_p$ and $H_d : \mathbb{Z}_p^5 \rightarrow \mathbb{Z}_p$, and parameters for the improved Groth-Sahai proof [16] and Dory [26].

IS.Convert. On input a signature σ , a message m , a public key pk^* , and a set of public keys $\mathbf{pk} = \{pk_1, \dots, pk_n\}$, with $pk_j = pk^*$ for some index $j \in [1, n]$, it converts the verification equation to:

$$\hat{e}(\sigma, g_2) = \hat{e}(H(m), \prod_{i=1}^n pk_i^{b_i}),$$

where $b_j = 1$ and $b_i = 0$ for all $i \neq j$. In order to maintain anonymity, it needs a zero-knowledge proof for all b_i , as well as σ and the index j . There are a few steps to fulfil this goal.

- (1) Commit pk_j . Compute a group element $g_0 = H_g(c, \mathbf{pk})$. Pick a random $\beta \in \mathbb{Z}_p$ and compute a commitment $C_{pk} = g_0^\beta pk_j = g_0^\beta \mathbf{pk}^{\vec{b}}$, where $\vec{b} = (b_1, \dots, b_n)$.
- (2) ZK Proof for σ . It generates the improved Groth-Sahai proof [16] π_{GS} for pairing product equation $\hat{e}(\sigma, g_2) \cdot \hat{e}(H(m), g_0^\beta) = \hat{e}(H(m), C_{pk})$ for witness σ, β .
- (3) ZK Proof for $\vec{b}$. It is the same as the incognito Schnorr signature. Denote $\pi_{\vec{b}} = (A, S, S_{pk}, T_1, T_2, \tau_x, \mu, v, \hat{t}, \vec{l}, \vec{r})$.
- (4) Compressing vectors. It is similar to the incognito Schnorr signature. Instead of using Bulletproof, it can use pairing-based ZK proof π'_b such as Dory [26].

Finally, it outputs the incognito signature $\sigma_I = (C_{pk}, \pi_{GS}, \pi'_b)$.

IS.Verify. On input an incognito signature $\sigma_I = (C_{pk}, \pi_{GS}, \pi'_b)$, a message m and a set of public key $\mathbf{pk}$, it outputs 1 if both π_{GS}, π'_b are valid and outputs 0 otherwise.

The security of the incognito BLS signature follows the security proof of the generic incognito signature.

THEOREM B.1. *The incognito BLS signature is strongly posterior anonymous against full key and signature exposure in the random oracle model.*

PROOF. It can be shown by the following lemmas.

LEMMA B.2. *π'_b is perfect honest-verifier zero-knowledge for the witness $\vec{b}$.*

PROOF. By picking random $x, w, y, d, \mu, v, \hat{t} \in \mathbb{Z}_p$ and $S, T_2, S_{pk} \in \mathbb{G}$, we can compute T_1, A from equation 5 and 8. π_D is perfect honest-verifier zero-knowledge Dory. By setting $(y, w) = H_w(A, S, S_{pk}, g_0, C_{pk})$ and $x = H_x(T_1, T_2, y, w)$, π'_b is zero-knowledge. $\square$

LEMMA B.3. *π_{GS} is perfect honest-verifier zero-knowledge for the witness σ, β .*

PROOF. Groth-Sahai proof is zero-knowledge for witness σ, β $\square$

The simulator $\mathcal{S}$ runs $\text{param} \leftarrow \text{Setup}(1^\lambda)$ and $\text{param}_I \leftarrow \text{IS.Setup}(\text{param})$. $\mathcal{S}$ runs $(sk_i, pk_i) \leftarrow \text{KeyGen}()$ using randomness ω_i for $i \in [1, n]$ and sets $\mathbf{pk} = (pk_1, \dots, pk_n)$. $\mathcal{S}$ sends $\text{param}, \text{param}_I, \mathbf{pk}$, and $\{\omega_i\}_{i=1}^n$ to the adversary $\mathcal{A}$.

The adversary $\mathcal{A}$ returns a message m^* , two indices i_0, i_1 , two signatures $\sigma_0^* \leftarrow \text{Sign}(m^*, sk_{i_0}), \sigma_1^* \leftarrow \text{Sign}(m^*, sk_{i_1})$ using randomness ω_0, ω_1 , and a set of public key $\mathbf{pk}^*$ for which $pk_{i_0}, pk_{i_1} \in \mathbf{pk} \cap \mathbf{pk}^*$.

$\mathcal{S}$ picks a random bit b and runs random group element $C_{pk} \in \mathbb{G}$. Since the Pedersen commitment is perfectly hiding, C_{pk} is indistinguishable from a real commitment of pk_{i_b} .

By the above lemmas, $\mathcal{S}$ can generate π_{GS} and π'_b that have no information about $\sigma_b^*, \vec{b}$ and j , and hence no information about i_b . Hence no PPT adversary can guess $b^* = b$ with probability larger than $1/2$. The incognito BLS signature is strongly anonymous against full key exposure. $\square$

Extension. Our generic incognito signature can also be applied to the pairing-based BBS/BBS+ signature [8], structure-preserving signatures, etc.

By using structure-preserving signatures [23] and the improved Groth-Sahai proof [16], it is possible to construct incognito signature from standard assumptions, without using random oracle.

C PROPERTIES OF ZERO-KNOWLEDGE ARGUMENT OF KNOWLEDGE

Perfect completeness. A prover with a witness w for $u \in \mathcal{L}_\tau$ can convince the verifier of this fact. *For any non-uniform polynomial time adversary $\mathcal{A}, (\mathcal{K}, \mathcal{P}, \mathcal{V})$ has perfect completeness if*

$$\Pr[(\tau, u, w) \notin \mathcal{R} \text{ or } \langle \mathcal{P}(\tau, u, w), \mathcal{V}(\tau, u) \rangle = 1]$$

$$\tau \leftarrow \mathcal{K}(1^\lambda), (u, w) \leftarrow \mathcal{A}(\tau) = 1.$$

Statistical witness-extended emulation. Given an adversary that produces an acceptable argument with probability ϵ , there exists an emulator that produces a similar argument with probability ϵ together with a witness w . *For any deterministic polynomial time prover $\mathcal{P}^*, (S, \mathcal{P}, \mathcal{V})$ has witness-extended emulation if there exists a polynomial time emulator $\mathcal{E}$ such that for any pair of interactive adversaries $\mathcal{A}_1$ and $\mathcal{A}_2$ such that*

$$\Pr[\mathcal{A}_1(tr) = 1 \mid \tau \leftarrow \mathcal{K}(1^\lambda), (u, s) \leftarrow \mathcal{A}_2(\tau),$$

$$tr \leftarrow \langle \mathcal{P}^*(\tau, u, s), \mathcal{V}(\tau, u) \rangle]$$

$$\approx \Pr[\mathcal{A}_1(tr) = 1 \wedge (tr \text{ is accepting} \Rightarrow (\tau, u, w) \in \mathcal{R}) \mid$$

$$\tau \leftarrow \mathcal{K}(1^\lambda), (u, s) \leftarrow \mathcal{A}_2(\tau), (tr, w) \leftarrow \mathcal{E}^O(\tau, u)],$$

where the oracle $O = \langle \mathcal{P}^*(\tau, u, s), \mathcal{V}(\tau, u) \rangle$ can rewind to some point and resume with new randomness for the verifier $\mathcal{V}$ from this point onward.

Public Coin. An argument of knowledge $(\mathcal{K}, \mathcal{P}, \mathcal{V})$ is called public coin if all messages sent from the verifier to the prover are chosen uniformly at random and independently of the prover's messages, i.e., the challenges correspond to the verifier's randomness ρ .

By using the Fiat-Shamir heuristic, a public coin interactive proof of knowledge can be converted into a non-interactive proof of knowledge.

Perfect Special Honest-Verifier Zero-Knowledge (SHVZK). A public coin argument of knowledge $(\mathcal{K}, \mathcal{P}, \mathcal{V})$ is a perfect SHVZK argument of knowledge for $\mathcal{R}$ if there exists a PPT simulator $\mathcal{S}$ such that for all pairs of interactive adversaries $\mathcal{A}_1$ and $\mathcal{A}_2$

$$\begin{aligned} & \Pr[(\tau, u, w) \in \mathcal{R} \wedge \mathcal{A}_1(tr) = 1 \mid \tau \leftarrow \mathcal{K}(1^\lambda), \\ & \quad (u, w, \rho) \leftarrow \mathcal{A}_2(\tau), tr \leftarrow \langle \mathcal{P}(\tau, u, w), \mathcal{V}(\tau, u; \rho) \rangle] \\ &= \Pr[(\tau, u, w) \in \mathcal{R} \wedge \mathcal{A}_1(tr) = 1 \mid \tau \leftarrow \mathcal{K}(1^\lambda), \\ & \quad (u, w, \rho) \leftarrow \mathcal{A}_2(\tau), tr \leftarrow \mathcal{S}(u, \rho)]. \end{aligned}$$

There are efficient standard techniques to convert an SHVZK argument into a public-coin full zero-knowledge argument (with a cheating verifier) when a common reference string is available.

D DETAILS OF THE INCOGNITO SCHNORR SIGNATURE

D.1 Details of the NIZKAoK π'_b

Sign. Using a random challenge $y \in \mathbb{Z}_p$, we want to prove that $\langle \vec{b}, \vec{a} \circ \vec{y}^n \rangle = 0$, $\langle \vec{b} - \vec{1}^n - \vec{a}, \vec{y}^n \rangle = 0$ and $\langle \vec{b}, \vec{1}^n \rangle = 1$, which is equivalent to:

$$w^2 \cdot \langle \vec{b}, \vec{1}^n \rangle + w \cdot \langle \vec{b} - \vec{1}^n - \vec{a}, \vec{y}^n \rangle + \langle \vec{b}, \vec{a} \circ \vec{y}^n \rangle = w^2,$$

for some random challenge $w \in \mathbb{Z}_p$. This first equality can be re-written as:

$$\langle \vec{b} - w \cdot \vec{1}^n, \vec{y}^n \circ (\vec{a} + w \cdot \vec{1}^n) + w^2 \cdot \vec{1}^n \rangle = w^2 - w^3 n + (w - w^2) \langle \vec{1}^n, \vec{y}^n \rangle.$$

We pick some random $\alpha, \rho, \zeta \in \mathbb{Z}_p$ and binding vectors $\vec{s}_a, \vec{s}_b \in \mathbb{Z}_p^n$ to compute

$$A = h^\alpha \vec{g}^{\vec{b}} \vec{h}^{\vec{a}}, \quad S = h^\rho \vec{g}^{\vec{s}_b} \vec{h}^{\vec{s}_a}, \quad S_{pk} = g_0^\zeta \mathbf{pk}^{\vec{s}_b}.$$

We compute the challenge $(y, w) = H_w(A, S, S_{pk}, g_0, C_{pk})$. Define the following polynomials:

$$\begin{aligned} l(X) &= (\vec{b} - w \cdot \vec{1}^n) + \vec{s}_b \cdot X, \\ r(X) &= \vec{y}^n \circ (\vec{a} + w \cdot \vec{1}^n + \vec{s}_a \cdot X) + w^2 \cdot \vec{1}^n, \\ t(X) &:= \langle l(X), r(X) \rangle = t_0 + t_1 \cdot X + t_2 \cdot X^2. \end{aligned}$$

It implies that $t_0 = w^2 - w^3 n + (w - w^2) \langle \vec{1}^n, \vec{y}^n \rangle$ and the coefficients $t_1, t_2 \in \mathbb{Z}_p$ can be computed as:

$$t_1 = \langle \vec{s}_b, \vec{y}^n \circ (\vec{a} + w \cdot \vec{1}^n) + w^2 \cdot \vec{1}^n \rangle + \langle \vec{b} - w \cdot \vec{1}^n, \vec{y}^n \circ \vec{s}_a \rangle, \quad t_2 = \langle \vec{s}_b, \vec{y}^n \circ \vec{s}_a \rangle.$$

We pick some random $\tau_1, \tau_2 \in \mathbb{Z}_p$ and compute:

$$T_1 = g^{\tau_1} h^{\tau_1}, \quad T_2 = g^{\tau_2} h^{\tau_2},$$

We compute the challenge $x = H_x(T_1, T_2, y, w)$. We compute $\vec{l} = l(x)$, $\vec{r} = r(x)$, $\hat{t} = t(x)$ and

$$\tau_x = \tau_2 x^2 + \tau_1 x, \quad \mu = \alpha + \rho x, \quad \nu = \beta + \zeta x.$$

Denote $\pi'_b = (A, S, S_{pk}, T_1, T_2, \tau_x, \mu, \nu, \hat{t}, \vec{l}, \vec{r})$.

Note that π'_b is of size $O(n)$. We now compress vectors $(\vec{l}, \vec{r})$ using the Bulletproof. Define $\vec{h}' := (h_1, h_2^{y^{-1}}, h_3^{y^{-2}}, \dots, h_n^{y^{-n+1}})$. Observe

that the ZK proof π'_b can be validated by:

$$\hat{t} = \langle \vec{l}, \vec{r} \rangle. \quad (4)$$

$$g^{\hat{t}} h^{\tau_x} = g^{w^2 - w^3 n + (w - w^2) \langle \vec{1}^n, \vec{y}^n \rangle} T_1^x T_2^{x^2} \quad (5)$$

$$h^\mu \vec{g}^{\vec{l}} \vec{h}'^{\vec{r}} = AS^x \vec{g}^{-w \cdot \vec{1}^n} \vec{h}'^{w \cdot \vec{y}^n + w^2 \cdot \vec{1}^n} \quad (6)$$

$$g_0^\nu \mathbf{pk}^{\vec{l}} = g_0^{\beta + \zeta x} \mathbf{pk}^{\vec{b} - w \cdot \vec{1}^n + \vec{s}_b x} = C_{pk} \mathbf{pk}^{-w \cdot \vec{1}^n} S_{pk}^x. \quad (7)$$

We compute $d = H_d(x, \tau_x, \mu, \nu, \hat{t})$ and combines Eq. 6 and 7 by:

$$\begin{aligned} h^\mu \vec{g}^{\vec{l}} \vec{h}'^{\vec{r}} (g_0^\nu \mathbf{pk}^{\vec{l}})^d &= AS^x \vec{g}^{-w \cdot \vec{1}^n} \vec{h}'^{w \cdot \vec{y}^n + w^2 \cdot \vec{1}^n} (C_{pk} \mathbf{pk}^{-w \cdot \vec{1}^n} S_{pk}^x)^d \\ &\Rightarrow g_0^{d\nu} h^\mu (\vec{g} \circ \mathbf{pk}^d)^{\vec{l}} \vec{h}'^{\vec{r}} = AS^x C_{pk}^{xd} S_{pk}^{xd} (\vec{g} \circ \mathbf{pk}^d)^{-w \cdot \vec{1}^n} \vec{h}'^{w \cdot \vec{y}^n + w^2 \cdot \vec{1}^n} \end{aligned} \quad (8)$$

We compute $P = (\vec{g} \circ \mathbf{pk}^d)^{\vec{l}} \vec{h}'^{\vec{r}}$ and obtain a Bulletproof [10] π_B for $(P, \hat{t})$ with witness $(\vec{l}, \vec{r})$ and generators $\vec{g} \circ \mathbf{pk}^d$ and $\vec{h}'$. Denote $\pi'_b = (A, S, S_{pk}, T_1, T_2, \tau_x, \mu, \nu, \hat{t}, \pi_B)$.

Verify. For $\pi'_b = (A, S, S_{pk}, T_1, T_2, \tau_x, \mu, \nu, \hat{t}, \pi_B)$, it computes $(y, w) = H_w(A, S, S_{pk}, g_0, C_{pk})$, $x = H_x(T_1, T_2, y, w)$ and checks if:

$$g^{\hat{t}} h^{\tau_x} = g^{w^2 - w^3 n + (w - w^2) \langle \vec{1}^n, \vec{y}^n \rangle} T_1^x T_2^{x^2}.$$

To verify π_B , it computes $d = H_d(x, \tau_x, \mu, \nu, \hat{t})$, $\vec{h}' = (h_1, h_2^{y^{-1}}, h_3^{y^{-2}}, \dots, h_n^{y^{-n+1}})$ and $P = AS^x C_{pk}^{xd} S_{pk}^{xd} (\vec{g} \circ \mathbf{pk}^d)^{-w \cdot \vec{1}^n} \vec{h}'^{w \cdot \vec{y}^n + w^2 \cdot \vec{1}^n} g_0^{-d\nu} h^{-\mu}$.

It checks the Bulletproof π_B with respect to $(P, \hat{t})$ and generators $\vec{g} \circ \mathbf{pk}^d$ and $\vec{h}'$.

D.2 Security Proofs

THEOREM D.1. *The incognito Schnorr signature has correctness.*

PROOF. We first check that:

$$\begin{aligned} & t(x) \\ &= \langle l(x), r(x) \rangle \\ &= \langle (\vec{b} - w \cdot \vec{1}^n) + \vec{s}_b \cdot x, \vec{y}^n \circ (\vec{a} + w \cdot \vec{1}^n + \vec{s}_a \cdot x) + w^2 \cdot \vec{1}^n \rangle \\ &= \langle \vec{b} - w \cdot \vec{1}^n, \vec{y}^n \circ (\vec{a} + w \cdot \vec{1}^n + \vec{s}_a \cdot x) \rangle + w^2 \underbrace{\langle \vec{b}, \vec{1}^n \rangle}_{=1} \\ &\quad - w^3 \underbrace{\langle \vec{1}^n, \vec{1}^n \rangle}_{=n} + x \underbrace{\langle \vec{s}_b, \vec{y}^n \circ (\vec{a} + w \cdot \vec{1}^n + \vec{s}_a \cdot x) + w^2 \cdot \vec{1}^n \rangle}_{=n} \\ &= \underbrace{\langle \vec{b}, \vec{y}^n \circ \vec{a} \rangle}_{=0} + \langle \vec{b}, w \vec{y}^n \rangle + \langle \vec{b} - w \cdot \vec{1}^n, \vec{y}^n \circ \vec{s}_a \cdot x \rangle \\ &\quad - w \langle \vec{1}^n, \vec{y}^n \circ (\vec{a} + w \cdot \vec{1}^n) \rangle + w^2 - w^3 n \\ &\quad + x \langle \vec{s}_b, \vec{y}^n \circ (\vec{a} + w \cdot \vec{1}^n) + w^2 \cdot \vec{1}^n \rangle + x^2 \underbrace{\langle \vec{s}_b, \vec{y}^n \circ \vec{s}_a \rangle}_{=t_2} \\ &= w \langle \vec{b}, \vec{y}^n \rangle - w \langle \vec{1}^n, \vec{y}^n \circ \vec{a} \rangle - w^2 \langle \vec{1}^n, \vec{y}^n \rangle + w^2 - w^3 n \\ &\quad + x \underbrace{\langle \vec{b} - w \cdot \vec{1}^n, \vec{y}^n \circ \vec{s}_a \rangle + x \langle \vec{s}_b, \vec{y}^n \circ (\vec{a} + w \cdot \vec{1}^n) + w^2 \cdot \vec{1}^n \rangle}_{=t_1 x} + t_2 x^2 \end{aligned}$$

$$\begin{aligned}
&= w \underbrace{\langle \vec{b} - \vec{1}^n - \vec{a}, \vec{y}^n \rangle}_{=0} + w \underbrace{\langle \vec{1}^n, \vec{y}^n \rangle - w^2 \langle \vec{1}^n, \vec{y}^n \rangle + w^2 - w^3 n}_{=t_0} \\
&\quad + t_1 x + t_2 x^2 \\
&= t_0 + t_1 x + t_2 x^2.
\end{aligned}$$

We have:

$$\begin{aligned}
g^{s_z} g_0^{s_{\beta} c} &= g^{r_z + c_z z} g_0^{c(r_{\beta} + c_{\beta} \beta)} = R_z(g^z g_0^{c \beta})^{c_z} \\
&= R_z(R_{\text{pk}} g_0^{c \beta})^{c_z} = R_z R^{c_z} C_{\text{pk}}^{c c_z}, \\
g^{\hat{t}} h^{\tau_x} &= g^{t_0 + t_1 x + t_2 x^2} h^{\tau_2 x^2 + \tau_1 x} = g^{t_0} (g^{t_1} h^{\tau_1})^x (g^{t_2} h^{\tau_2})^{x^2} \\
&= g^{w^2 - w^3 n + (w - w^2) \langle \vec{1}^n, \vec{y}^n \rangle} T_1^x T_2^{x^2}, \\
P &= [A S^x C_{\text{pk}}^d S_{\text{pk}}^{x d} (\vec{g} \circ \text{pk}^d)^{-w} \vec{h}'^{w \cdot \vec{y}^n + w^2 \cdot \vec{1}^n} g_0^{-v} h^{-\mu} \\
&= (\vec{g} \circ \text{pk}^d)^{\vec{l}} \vec{h}'^{\vec{r}}.
\end{aligned}$$

□

LEMMA D.2. $\pi_{\vec{b}}$ is perfect honest-verifier zero-knowledge for the witness $\vec{b}$.

PROOF. By picking random $x, w, y, \mu, v \in \mathbb{Z}_p$, $\vec{l}, \vec{r} \in \mathbb{Z}_p^n$ and $S, T_2 \in \mathbb{G}$, we can compute $T_1, A, \hat{t}, S_{\text{pk}}$ from equation 5-7. By setting $(y, w) = H_w(A, S, S_{\text{pk}}, g_0, C_{\text{pk}})$ and $x = H_x(T_1, T_2, y, w)$, $\pi_{\vec{b}}$ is zero-knowledge. □

LEMMA D.3. $\pi'_{\vec{b}}$ is perfect honest-verifier zero-knowledge for the witness $\vec{b}$.

PROOF. By picking random $x, w, y, d, \mu, v, \hat{t} \in \mathbb{Z}_p$ and $S, T_2, S_{\text{pk}} \in \mathbb{G}$, we can compute T_1, A from equation 5 and 8. π_B is perfect honest-verifier zero-knowledge Bulletproof. By setting $(y, w) = H_w(A, S, S_{\text{pk}}, g_0, C_{\text{pk}})$ and $x = H_x(T_1, T_2, y, w)$, $\pi'_{\vec{b}}$ is zero-knowledge. □

LEMMA D.4. $\pi_{\vec{b}}$ is computational witness extended emulation for the witness $\vec{b}$, such that $\vec{b}$ is a binary vector with $b_j = 1$ for only one $j \in [1, n]$ and C_{pk} is a commitment of pk_j .

PROOF. We consider that $\pi_{\vec{b}}$ is verified by Eq. 5-7. Suppose that there are three sets of transcript $(A, S, T_1, T_2, v, \tau_{x,k}, \mu_k, \hat{t}_k, \vec{l}_k, \vec{r}_k)$ for challenges $x_k, k \in [1, 3]$. By using the linear combination of Eq. 5, we can compute $(t_1, t_2, \tau_1, \tau_2)$ such that:

$$T_1 = g^{t_1} h^{\tau_1}, \quad T_2 = g^{t_2} h^{\tau_2}.$$

By Eq. 5, if $\hat{t} \neq w^2 - w^3 n + (w - w^2) \langle \vec{1}^n, \vec{y}^n \rangle + t_1 x + t_2 x^2$ for any transcript, it yields a discrete logarithm relation between g and h . If not, then for all w challenges and 3 distinct challenges $X = x_k, k \in [1, 3]$, we have a degree 2 polynomial of X :

$$w^2 - w^3 n + (w - w^2) \langle \vec{1}^n, \vec{y}^n \rangle + t_1 X + t_2 X^2 - \langle l(X), r(X) \rangle.$$

By Eq. 4, the above polynomial has at least 3 distinct roots (x_1, x_2, x_3) . Hence it is a zero polynomial, i.e., $w^2 - w^3 n + (w - w^2) \langle \vec{1}^n, \vec{y}^n \rangle + t_1 X + t_2 X^2 = \langle l(X), r(X) \rangle$.

By using two sets of transcripts with different x_i , we can use the linear combination of Eq. 6 to compute $(\alpha, \vec{b}, \vec{a}, \rho, \vec{s}_b, \vec{s}_a)$ such that:

$$A = h^\alpha \vec{g}^{\vec{b}} \vec{h}^{\vec{a}}, \quad S = h^\rho \vec{g}^{\vec{s}_b} \vec{h}^{\vec{s}_a}. \quad (9)$$

If for any other set of challenges (x, y, w) , the extractor can compute a different representation of A or S , then this yields a non-trivial discrete logarithm relation between independent generators $h, \vec{g}, \vec{h}$ which contradicts the discrete logarithm relation assumption. Putting Eq. 9 back to Eq. 6, we can see that for all challenges (x, y, w) :

$$\mu = \alpha + \rho x, \quad \vec{l} = \vec{b} - w \cdot \vec{1}^n + \vec{s}_b x, \quad \vec{r} = \vec{y}^n \circ (\vec{a} + w \cdot \vec{1}^n + \vec{s}_a \cdot x) + w^2 \cdot \vec{1}^n.$$

The above holds for three distinct challenges of x and w , and n distinct challenges of y . Consider the terms without x or x^2 , we have:

$$\langle \vec{b} - w \cdot \vec{1}^n, \vec{y}^n \circ (\vec{a} + w \cdot \vec{1}^n) + w^2 \cdot \vec{1}^n \rangle = w^2 - w^3 n + (w - w^2) \langle \vec{1}^n, \vec{y}^n \rangle.$$

It can be rewritten as $w^2 \cdot \langle \vec{b}, \vec{1}^n \rangle + w \cdot \langle \vec{b} - \vec{1}^n - \vec{a}, \vec{y}^n \rangle + \langle \vec{b}, \vec{a} \circ \vec{y}^n \rangle = w^2$. It implies that $\langle \vec{b}, \vec{a} \circ \vec{y}^n \rangle = 0$, $\langle \vec{b} - \vec{1}^n - \vec{a}, \vec{y}^n \rangle = 0$ and $\langle \vec{b}, \vec{1}^n \rangle = 1$.

Similarly, given two distinct challenges of x over Eq. 7, we can extract β, ζ such that:

$$C_{\text{pk}} = g_0^\beta \text{pk}^{\vec{b}}, \quad S_{\text{pk}} = g_0^\zeta \text{pk}^{\vec{s}_b}$$

Hence C_{pk} is a commitment of pk_j . □

LEMMA D.5. $\pi'_{\vec{b}}$ is computational witness extended emulation for the witness $\vec{b}$.

PROOF. Denote $\pi'_b = (A, S, S_{\text{pk}}, T_1, T_2, \tau_x, \mu, v, \hat{t}, \pi_B)$. First, note that d is computed from the hash of x , x is computed from the hash of (y, w) , (y, w) is computed from the hash of g_0 , and g_0 is computed from the hash of pk . Hence the generators $\vec{g} \circ \text{pk}^d$ and $\vec{h}'$ has no known discrete logarithm relation. By the computational witness extended emulation property of Bulletproof, we can extract $(\vec{l}, \vec{r})$ from π_B , such that Eq. 8 and Eq. 4 hold.

Given two sets of transcript of π_B with different d_1, d_2 , we have both Eq. 6 and Eq. 7 hold. By lemma D.4, the witness $\vec{b}$ can be extracted. □